



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Dipartimento di

Corso di Laurea in

Titolo della tesi
anche su più
righe

Relatore:
Chiar.mo Prof.
Nome Cognome

Correlatore:
Chiar.mo Prof.
Nome Cognome

Presentata da:
Nome Cognome

Sessione mese anno
Anno Accademico 20xx/20xx

(DA FARE ALLA FINE)

5 parole chiave per caratterizzare il contenuto della dissertazione:
(se non ti piacciono così sparse puoi anche semplicemente scriverle su una riga
sola)

parola 5

parola 4

parola 3

parola 2

Parola 1

*La dedica
anche quella se vuoi
su più righe*

Abstract

La letteratura economica moderna, in particolare i modelli di crescita endogena, suggerisce che lo sviluppo di lungo periodo di un sistema dipende soprattutto dalla capacità di generare e diffondere innovazione. In questo contesto, le infrastrutture di comunicazione non sono più semplici canali passivi, ma diventano attori centrali nel processo di crescita.

In campi come Industria 4.0 (ma anche automotive, audio/video, sanità ecc..) riscontriamo la necessità di migliorare l'accesso alle informazioni, dato che ciò porta a:

- *Abbassare i costi*
- *Aumentare l'efficienza*
- *Garantire sicurezza*

Questi sono gli obiettivi principali dei processi di convergenza IT/OT a cui stiamo assistendo oggi, che puntano ad un cambio di paradigma, passando da gerarchie rigide (si veda ISA 95) a modelli "piatti" che favoriscono l'interoperabilità e l'interazione diretta tra i diversi livelli della piramide industriale, e ci permettono di pensare alle informazioni come ad un flusso unico e continuo.

Per supportare questa convergenza è necessario far convivere le caratteristiche intrinseche dei due mondi, e gestire aspetti come:

- *La presenza di più flussi di informazioni, ognuno con il proprio grado di "criticità"*
- *Comunicazione deterministica, e cosa ciò comporta nel "mondo" IT, ricordando che la semantica di comunicazione che prevale nelle reti è di tipo best-effort*

Nei prossimi capitoli verranno presentate alcune delle tecnologie fondamentali per supportare questa transizione, e come queste possono essere usate in situazioni reali.

Indice

Introduzione	1
1 Caratteristiche e obiettivi di TSN & Wi-Fi 7	3
1.1 Sincronizzazione temporale	3
1.2 Latenza Limitata e 0 perdite causate da congestione	4
1.2.1 Algoritmi di accodamento	5
1.3 Affidabilità	9
1.3.1 FRER - Frame Replication and Elimination for Reliability	9
1.3.2 PCR - Path Control and Reservation	10
1.3.3 PSFP - Per-Stream Filtering and Policing	11
1.4 Gestione risorse e rete	11
1.4.1 Modello Completamente Distribuito	11
1.4.2 Modello a Rete Centralizzata e Utente Distribuito	11
1.4.3 Modello Completamente Centralizzato	12
1.4.4 Allocazione delle risorse	13
1.5 Wi-Fi 7	14
1.5.1 Supporto al TSN	17
2 Stack 802.11 in Linux e supporto al TSN	19
2.1 Stack 802.11	19
2.2 Mappare servizi TSN in Linux	22
2.2.1 Sincronizzazione temporale: Linux PTP	22
2.3 Latenza Limitata e 802.1Q: VLAN, TAPRIO, Frame Preemption	28
2.3.1 tc	28
2.3.2 Frame Preemption	30
2.4 Affidabilità: FRER	31
2.5 Gestione delle risorse	34
3 Prototipo	37
3.1 Obiettivi e assunzioni	37

3.2	Struttura	39
3.3	Motivazioni e scelte progettuali	41
3.3.1	FRER	41
3.3.2	TAPRIO	43
3.3.3	PTP	44
3.3.4	Unidirezionalità	47
3.4	Test e Considerazioni	47
3.4.1	FRER	47
3.4.2	PTP	49
3.4.3	TAPRIO	51
3.4.4	Conclusioni	57
	Riferimenti bibliografici	59
	Appendici	
	Appendice A Ambiente docker per test Linux PTP	
	Appendice B Macchina Host e Ambiente	
	Appendice C Configurazione strumenti wireless	
	Appendice D Tool Linux per il networking	
D.1	veth	
D.2	network namespace	

Introduzione

A supportare la maggior parte delle infrastrutture di rete troviamo i protocolli 802.3 e 802.11, ethernet standard e WiFi rispettivamente, protocolli che si occupano di gestire il livello 1 (livello fisico) e il livello 2 (livello di data link) della pila protocollare.

attraverso una semantica di comunicazione di tipo *best-effort* essi non forniscono garanzie sulla ricezione dei frame, ma il compito viene delegato ai livelli superiori. Per questi motivi viene introdotto da parte dell' 802.1 Time-Sensitive Networking Task Group, a partire dagli standard definiti dal protocollo 802.1Q, un insieme di standard e strumenti per fornire comunicazioni mediante reti 802 in grado garantire il trasferimento dei pacchetti in maniera deterministica e affidabile. [27]

Concetti preliminari

- **VLAN:** Raggruppamento logico di risorse di rete connesse ad una porta su uno switch, permettono suddivisione del dominio di broadcast e segmentazione di una LAN
- **Bridge:** Un bridge è un dispositivo che opera a livello 2 OSI e permette di connettere e filtrare traffico tra 2 o più segmenti di rete. Come le VLAN favoriscono la segmentazione, diminuendo quindi il dominio di collisione, sono fondamentali per supportare l'interoperabilità tra diverse tecnologie (Ethernet & Wifi)
- **Access Link:** Un access link appartiene, e trasporta traffico appartenente, ad una sola VLAN, il traffico è in formato "nativo" ovvero non contiene informazioni riguardo la VLAN ed il collegamento avviene tra l'end device e l'access port. Il dispositivo collegato ad un access port non ha visibilità della VLAN ma "vede" un unico dominio di broadcast

- **Trunk Link:** Un trunk link appartiene, e trasporta traffico appartenente, a più VLAN, connette quindi dispositivi che "comprendono" il concetto di VLAN, solitamente uno switch o un router
- **VLAN tagging:** permette allo switch di aggiungere ai frame (Livello 2) informazioni riguardo la VLAN, così facendo altri dispositivi in grado di "capire" le VLAN sapranno cosa fare con quel frame, il tag verrà rimosso prima di mandare il frame all'end device [19]
- **SDN:** Insieme di meccanismi che permette di disaccoppiare il control plane (scelte di routing, topologia di rete, gestione policy) dal data plane, un approccio del genere porta ad infrastruttura programmabile e scalabile centralizzata [4]

Capitolo 1

Caratteristiche e obiettivi di TSN & Wi-Fi 7

1.1 Sincronizzazione temporale

Uno dei requisiti fondamentali per una rete deterministica è una visione condivisa e universale del tempo, TSN utilizza lo standard IEEE 802.1AS o gPTP - *Generalized Precision Time Protocol*, esso definisce un meccanismo di sincronizzazione del clock basato su IEEE 1588.

Il Precision Time Protocol una volta eletto un GM - Grandmaster, ovvero il dispositivo con la sorgente temporale più precisa, utilizza frame Ethernet per sincronizzare il clock dei dispositivi della rete utilizzando un approccio Grandmaster - Slave mediante messaggi di Sync, al quale gli Slave risponderanno con richieste per informazioni temporali aggiuntive, a queste il GM risponderà con messaggi di Follow-Up e Delay-Resp che permettono agli Slave di calcolare la differenza dei loro orologi rispetto a quella del GM e avere quindi una visione condivisa del tempo.

Per estendere i servizi offerti da 802.1AS in reti wireless sarà necessario l'utilizzo di protocolli come TM - *Timing Measurement* o FTM - *Fine Timing Measurement* che nascono come standard per misurare con precisione la distanza tra un STA e un AP sfruttando RTT, esistono in letteratura [15] soluzioni che propongono un mapping tra gPTP e FTM. I timestamp ottenuti tramite lo scambio di messaggi IEEE 802.1AS sul lato cablato vengono correlati con le misurazioni del tempo di volo, ToF - *Time of Flight*, effettuate tramite il protocollo FTM sul lato wireless.

Questo processo permette di compensare le variazioni del mezzo trasmissivo e di estendere il dominio di sincronizzazione TSN ai nodi mobili.

1.2 Latenza Limitata e 0 perdite causate da congestione

Uno degli obiettivi di TSN è quello di garantire bassa latenza e azzerare le perdite di frame causate dal congestionamento della rete, per rispettare questi vincoli si utilizzano diversi meccanismi basati sui concetti di schedulazione, modellazione del traffico e prelazione dei frame.

Se nel paradigma *best-effort* abbiamo una semantica di gestione del medium e dei nodi di inoltro del tipo *“Fai ciò che dice il pacchetto appena lo ricevi”*, [6] in TSN la necessità di determinismo e affidabilità (si veda la figura 1.1) porta ad una semantica: *“Fai ciò che dice il pacchetto quando lo dice il pacchetto”*.

Alla base di tutti gli standard definiti ritroviamo il concetto di *classe di traffico*, che permette di suddividere per l'appunto, il traffico, in diversi livelli di priorità, utilizzati poi dagli algoritmi di accodamento 1.2.1.

Un nodo di inoltro è un dispositivo che riceve un frame su una data porta in ingresso e lo invia su una (o più) porte in uscita, in TSN ad ogni porta in uscita è associata una coda, la quale mappa il concetto di classe di traffico, essa dovrà essere gestita in maniera opportuna: un dato algoritmo dovrà selezionare il frame *“giusto”* e inoltrarlo.

Un principio cardine nella gestione delle code è quello del *“mai un buffer vuoto”* [6] che porta al seguente requisito per una rete TSN:

“Per ogni flusso che attraversa un nodo di inoltro, viene bufferizzata una quantità di dati sufficiente a garantire che non venga mai persa un’opportunità di trasmissione per quel flusso, a meno che la sorgente stessa non rallenti o si interrompa”

Idealmente data una coda vorremmo che la frequenza dei pacchetti in ingresso fosse uguale a quella dei pacchetti in uscita, dato che se ciò non accade per una durata temporale sufficiente potremmo riscontrare perdite causate dalla congestione [12], che TSN cerca di abbattere.

Frequenze *“disaccoppiate”* sono causate da un jitter elevato che a sua volta può essere causato da:

- Variazioni nel processamento per l’inoltro del frame
- Algoritmi di accodamento che causano *“raffiche”* o *“intervalli”* nella trasmissione dei frame
- Sorgenti di trasmissione non coordinate

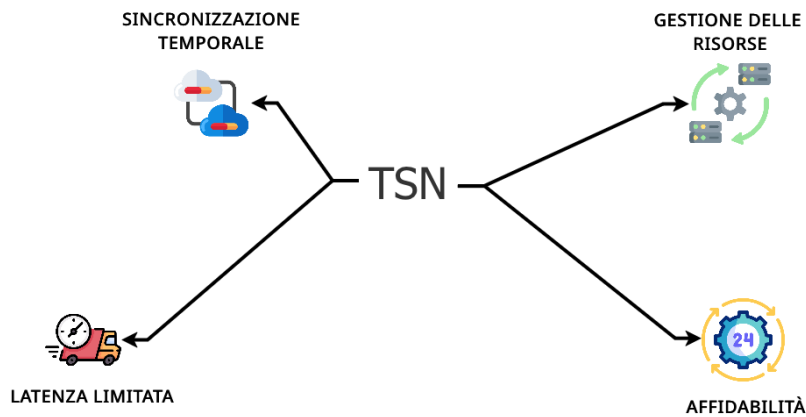


Figura 1.1: Obiettivi principali di TSN

1.2.1 Algoritmi di accodamento

Tutti gli algoritmi che verranno presi in esame appartengono allo standard 802.1, principalmente 802.1Q, Bridges e Bridges Networks, dove viene introdotto il concetto di VLAN tagging [8], questo meccanismo e, nello specifico i 3 bit di PCP - *Priority Code Point* permettono di supportare l'astrazione di classe di traffico.

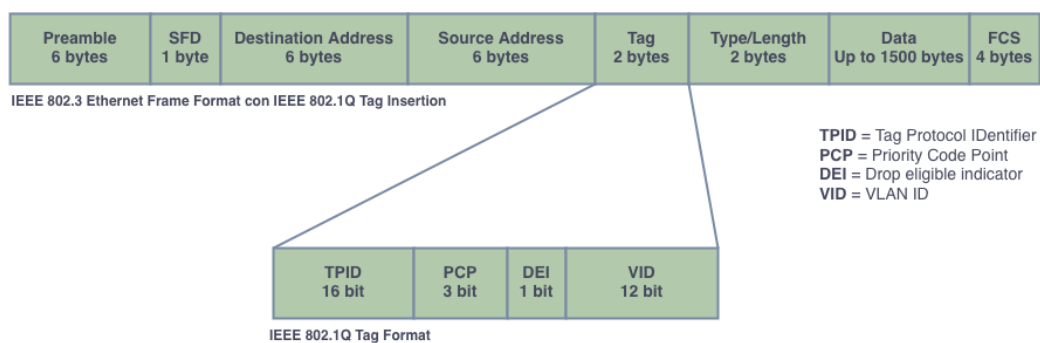


Figura 1.2: IEEE 802.3 Ethernet Frame Format con IEEE 802.1Q Tag Insertion

Come è possibile osservare in figura 1.2 nel campo Tag introdotto dallo standard 802.1Q sono presenti:

1. **TPID - Tag Protocol Identifier:** 16 bit che identificano il frame come frame VLAN-tagged, il valore è 0x8100
2. **PCP - Priority Code Point:** 3 bit che definiscono la priorità del frame riferendosi ad i CoS - *Class of Service* definiti nello standard IEEE 802.1p

3. **DEI - Drop Eligible Indicator:** Originariamente denominato CFI - *Canonical Format Indicator*, 1 bit che indica se il frame può essere eliminato in caso di congestione
4. **VID - VLAN Identifier:** 12 bit che identificano la VLAN a cui il frame appartiene

TAS - Time Aware Shaper

IEEE 802.1Qbv associa ad ogni classe di traffico presente su una porta in uscita una pianificazione ciclica definita dalla GCL - *Gate Control List*, in cui vengono specificati gli istanti temporali in cui una determinata classe può essere trasmessa, e quindi il gate è aperto.

Questo approccio “consapevole del tempo” richiede l'utilizzo di un clock condiviso che permette di operare con una granularità fino a 1 ns, ciò però è piuttosto oneroso a livello computazionale e infrastrutturale, in reti in cui è necessaria flessibilità, scalabilità ed è presente un traffico sporadico e non programmato, è spesso impraticabile, per questi casi introdurremo successivamente ATS.

TAS inoltre è in grado di gestire l'apertura di molteplici gate in contemporanea, favorendo il traffico critico, e implementa meccanismi di previsione per capire se riuscirà a inviare l'intero frame prima della chiusura del gate.

CBS - Credit Base Shaper

IEEE 802.1Qav, introdotto inizialmente per gestire i flussi multimediali si pone l'obiettivo principale di linearizzare il traffico, andando a ridurre le raffiche e gli intervalli, utilizzando un modello a credito. Ad ogni coda è associato un credito che viene accumulato quando questa è inattiva, e consumato quando avviene la trasmissione.

CQF - Cyclic Queuing Forwarding

IEEE 802.1Qch come TAS utilizza una pianificazione ciclica per la trasmissione, ma quest'ultima avviene anche per le porte in ingresso, il risultato è un pattern trasmissivo regolare senza le complicazioni introdotte da TAS, in quanto i frame si “spostano in gruppi”.

Frame Preemption

802.1Qbu e 802.3br permettono di definire due tipi di code:

- **Express:** I frame in queste code possono causare la prelazione di frame in code *preemptable*
- **Preemptable:** La trasmissione dei frame appartenenti a queste code può essere prelazionata per favorire traffico prioritario, la trasmissione riprenderà dopo la fine dei pacchetti critici

In figura 1.3 è possibile osservare l'architettura che supporta la *frame preemption*, il componente principale è MAC Merge layer, definito dallo standard IEEE 802.3br-2016 "Interspersing Express Traffic" [9], come un sottolivello opzionale che permette di alternare traffico espresso con traffico prelazionabile utilizzando un *preemptable Media Access Control* - pMAC ed un *express Media Access Control* - eMAC, il MAC Client è il livello che si occupa di definire quali pacchetti devono essere mandati all'eMAC o pMAC.

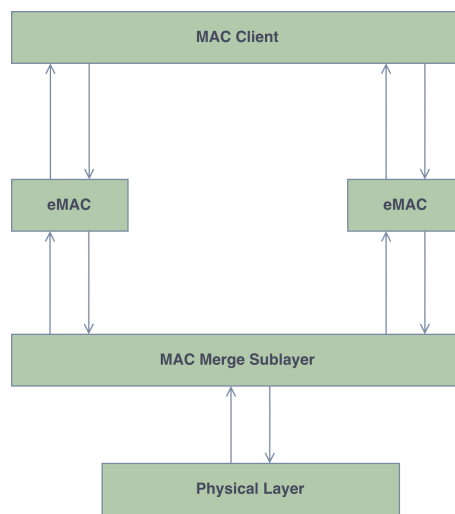


Figura 1.3: Relazioni tra MAC Merge Layer ed altri sottolivelli

Quando necessario, il MAC Merge layer consente di frammentare un frame prelazionabile, inserire il frame critico e riprendere successivamente la trasmissione del traffico interrotto.

L'utilizzo di questi meccanismi di prelazione riduce l'interferenza apportata dal traffico non critico in una rete deterministica, questo studio [7] mostra come sia possibile ottenere un miglioramento di circa il 91% sulla latenza massima e del 94,5% sul jitter.

In figura 1.4 è mostrata la differenza nella gestione del traffico con e senza frame preemption, si può notare come senza questo meccanismo, il traffico critico subisce un ritardo causato dalla trasmissione di frame non critici, mentre con la prelazione,

il traffico critico viene trasmesso immediatamente a meno di un delay d introdotto dai meccanismi di prelazione.

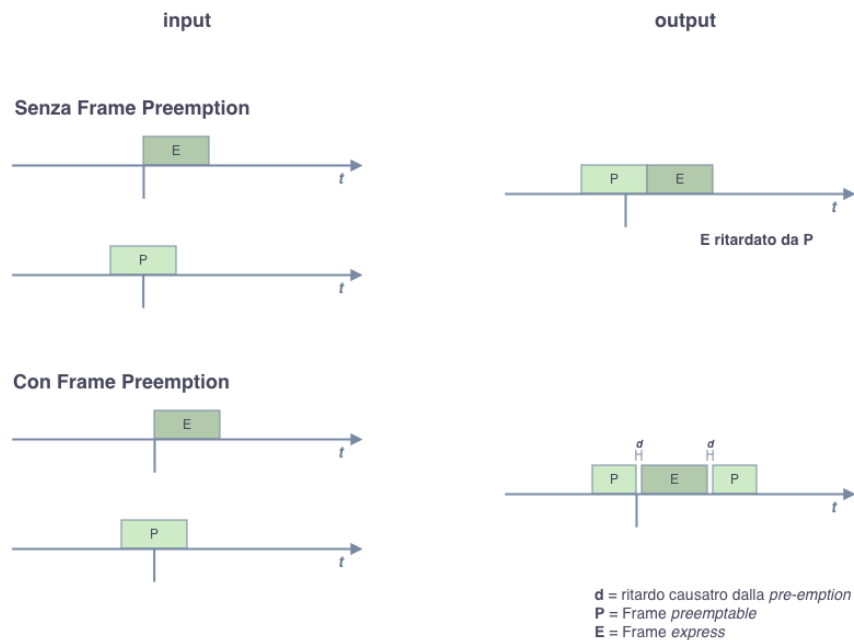


Figura 1.4: Differenza nella gestione del traffico con e senza frame preemption

ATS - Asynchronous Traffic Shaping

Nella descrizione di TAS sono state evidenziate le criticità in contesti dove il traffico non è periodico, IEEE 802.1Qcr si pone l'obiettivo di fornire modellazione e pianificazione del traffico senza la necessità di un elevato accoppiamento temporale (clock condiviso), sfruttando un approccio event-driven [28], a ogni pacchetto in ingresso viene assegnato un tempo di trasmissione. Ciò avviene tramite una macchina a stati che mostrata in figura 1.5, descrive i diversi stati possibili e gli eventi che possono scatenare una transizione tra di essi.

I 2 principali protocolli sono:

- **UBS - Urgency Based Scheduler:** Garantisce priorità al traffico critico
- **TBE - Token Bucket Emulation:** Garantisce una velocità media di trasmissione ed evita raffiche

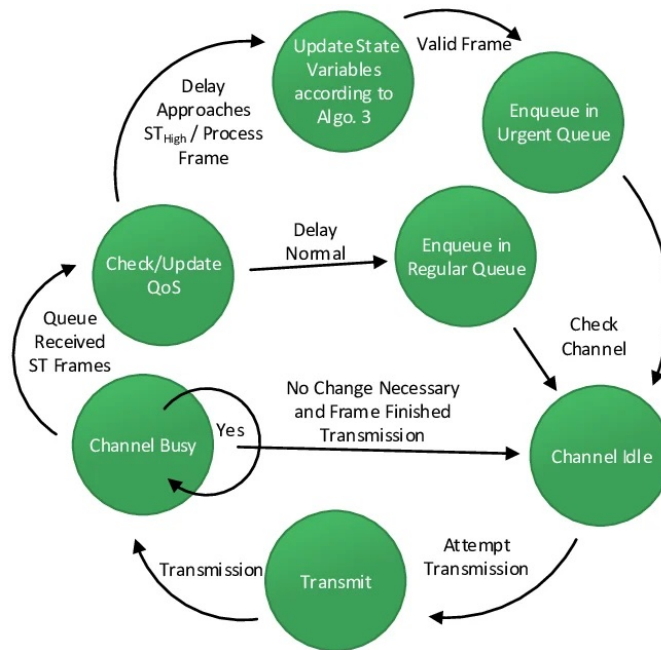


Figura 1.5: Macchina a stati di ATS, fonte: [28]

Questo algoritmo è ideale per situazioni ibride in cui coesistono sia traffico periodico che sporadico, per una comparazione dettagliata tra ATS e TAS si veda [18]

1.3 Affidabilità

Un elevato grado di affidabilità è richiesto per reti TSN, non essendo possibile basarsi su ritrasmissioni dei pacchetti data la natura *time-sensitive* del traffico, si introducono diversi meccanismi:

1.3.1 FRER - Frame Replication and Elimination for Reliability

Lo standard 802.1CB, permette di replicare i frame attraverso diversi percorsi, successivamente eliminando i duplicati. Ogni flusso duplicato viene chiamato *member stream*, all'interno di esso frame duplicati verranno identificati tramite IRF - *Individual Recovery Function*, *member stream* contenenti le stesse informazioni vengono chiamati *compound stream*. SRF - *Stream Recovery Function* si occupa invece di assegnare gli ID di sequenza prima che i frame vengano replicati. Successivamente i flussi replicati vengono eliminati a destinazione tramite uno di questi due algoritmi:

- **MAR - Match Recovery Algorithm:** Si basa sulla memorizzazione del numero di sequenza più elevato ricevuto fino a quel momento, se il nodo riceve un pacchetto con stesso identificatore lo scarta, altrimenti lo accetta e aggiorna la sua memoria
- **VRA - Vector Recovery Algorithm:** A differenza di MAR definisce un intervallo di numeri di sequenza, all'interno della quale nuovi frame sono accettati, i duplicati sono cancellati, frame non presenti nell'intervallo vengono scartati anche se nuovi

La scelta tra i due algoritmi risiede nello scenario che si prende in considerazione, MAR richiede meno memoria e risorse computazionali, ma allo stesso momento è meno flessibile rispetto a VRA in quanto richiede un flusso intermittente, sono definiti tali i flussi, i cui numeri di sequenza differiscono al massimo di uno. Per un'analisi più approfondita si esamini [14]

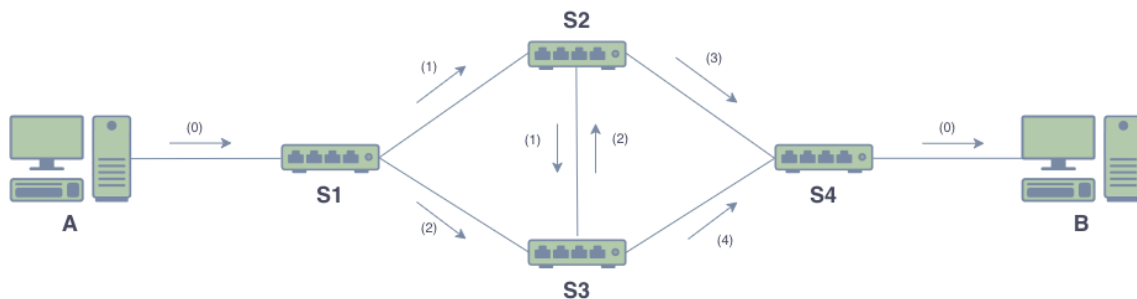


Figura 1.6: Flusso da host A ad host B mediante switch S1-S4

In figura 1.6 è mostrato un esempio di utilizzo di FRER, in cui il flusso da A a B viene replicato in S1 dove SRF assegna gli ID di sequenza ad i *member stream*, i frame vengono inviati su due percorsi diversi, e successivamente eliminati in S4 tramite IRF mediante un algoritmo a scelta tra MAR o VRA.

1.3.2 PCR - Path Control and Reservation

IEEE 802.1Qca si basa sull'estensione del protocollo IS-IS - *Intermediate System to Intermediate System* [24], permette un controllo esplicito del percorso, a differenza di protocolli tradizionali che si basano sul percorso più breve, la possibilità di allocare porzioni di banda e riservare risorse lungo percorsi selezionati e inoltre di gestire la ridondanza e la resilienza dei nodi.

1.3.3 PSFP - Per-Stream Filtering and Policing

IEEE 802.1Qci definisce protocolli e procedure volti a proteggere le reti filtrando, limitando e programmando i dati in ingresso a livello del singolo flusso [13].

1.4 Gestione risorse e rete

Il controllo della rete e la gestione delle risorse sono aspetti cruciali per garantire l'allocazione efficiente della banda, la priorità del traffico e il rispetto dei requisiti di QoS - *Quality of Service*. Per gestire le prenotazioni dei flussi e configurare la rete, gli standard TSN (in particolare IEEE 802.1Qcc, che estende lo Stream Reservation Protocol) definiscono tre opzioni o modelli architetturali principali [22].

1.4.1 Modello Completamente Distribuito

In questo modello non vi è alcuna entità di gestione centrale. Come è possibile osservare in figura 1.7 la rete viene configurata in modo completamente distribuito: i dispositivi finali (Talker e Listener) e i bridge (switch TSN) negoziano direttamente tra loro la registrazione e la prenotazione delle risorse (code, buffer, banda) scambiandosi U/NCI - *User-to-Network Configuration Information*, che viaggia in modalità *hop-by-hop* tra i nodi della rete. In questo modello il compito della UNI - *User-Network Interface* è quello di agire come confine tra dispositivo finale e bridge.

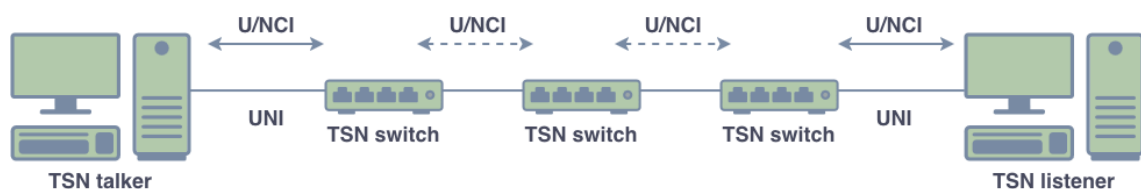


Figura 1.7: Modello Completamente Distribuito

1.4.2 Modello a Rete Centralizzata e Utente Distribuito

In questo modello, come mostrato in figura 1.8, i dispositivi finali inviano le loro specifiche esigenze di rete ai bridge di confine tramite l'interfaccia UNI. La rete, a questo punto, utilizza un'entità centrale chiamata CNC - *Centralized Network Configuration* che riceve questi requisiti e si occupa di configurare tutti i bridge della rete per soddisfare le specifiche.

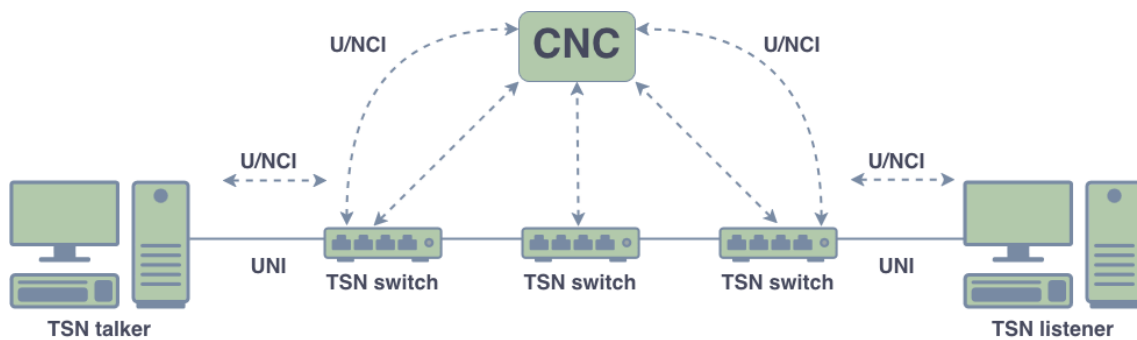


Figura 1.8: Modello a Rete Centralizzata e Utente Distribuito

1.4.3 Modello Completamente Centralizzato

Questo è il modello più avanzato per reti complesse, è possibile notare in figura 1.9 come qui vengano affiancate due entità software centralizzate:

- **CUC - Centralized User Configuration:** È un'applicazione che funge da proxy per i dispositivi finali. Comunica con essi, raccoglie i requisiti di comunicazione deterministica di cui hanno bisogno e formula una richiesta unica da passare al CNC
- **CNC - Centralized Network Configuration:** Riceve le richieste dal CUC, ha una visione globale della rete e svolge due compiti fondamentali: calcola i percorsi fisici (routing) e le schedulazioni end-to-end per ogni flusso, e infine applica queste configurazioni all'interno dei bridge TSN

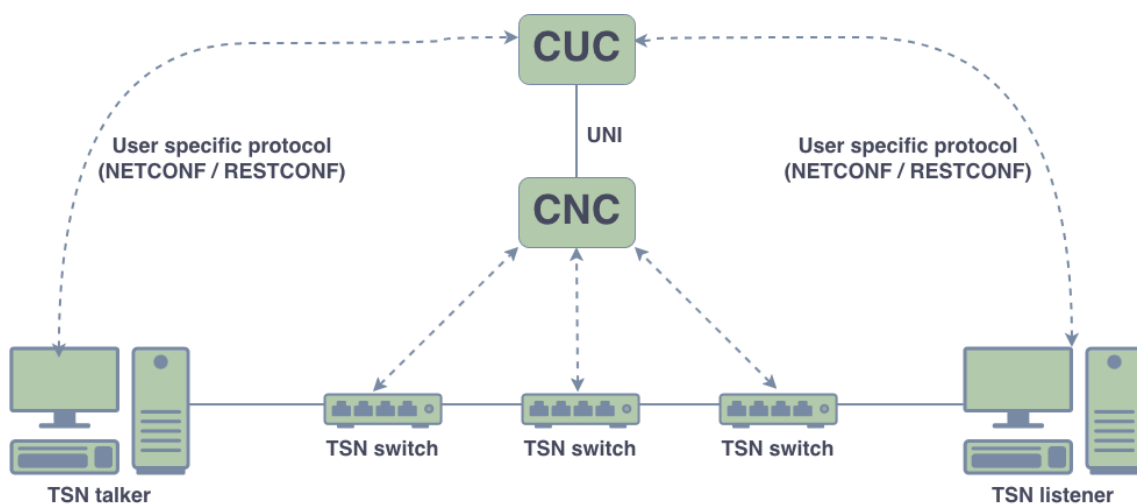


Figura 1.9: Modello Completamente Centralizzato

1.4.4 Allocazione delle risorse

L'ecosistema TSN si avvale di una suite di standard specifici per modellare e allocare le risorse

- **IEEE 802.1Qcp - Modello Dati YANG:** È il cuore della standardizzazione. Fornisce un linguaggio di modellazione unificato (YANG) per descrivere i dati di configurazione, lo stato dei dispositivi e le operazioni. Utilizzando protocolli di gestione di rete sicuri come NETCONF o RESTCONF, i controller possono inviare queste configurazioni standardizzate a dispositivi di produttori diversi, eliminando il "vendor lock-in"
- **IEEE 802.1CS - Link-Local Registration Protocol:** Ottimizza la replica dei database di registrazione a livello locale (da un capo all'altro di un singolo collegamento). Rispetto ai vecchi protocolli, LRP è in grado di gestire database molto più ampi (nell'ordine di 1 Mbyte), rendendo la gestione distribuita molto più efficiente
- **IEEE P802.1Qdd - Resource Allocation Protocol:** Sfrutta LRP per supportare la prenotazione dinamica delle risorse (per flussi unicast e multicast) all'interno del modello Completamente Distribuito. È nato per superare i limiti di scalabilità del vecchio protocollo MSRP
- **IEEE P802.1Qdj:** Uno standard in via di sviluppo che mira a chiarire e definire in modo preciso le funzioni specifiche delle due entità centrali, CUC e CNC, e a potenziare l'interfaccia di comunicazione UNI tra di esse

Attualmente, la ricerca e l'industria si concentrano fortemente sul modello completamente centralizzato, poiché si sposa perfettamente con i concetti del Software Defined Networking. In questo approccio, concetti come il TSSDN - *Time-Sensitive Software Defined Network* utilizzano un controller SDN per fungere da CNC, separando nettamente il piano dati da quello di controllo [23].

Il flusso di lavoro operativo si articola in quattro fasi principali:

1. *Scoperta della Topologia:* Il CNC esplora la rete utilizzando protocolli come LLDP (Link Layer Discovery Protocol) per ottenere una visione globale di tutti i nodi, i collegamenti e le risorse disponibili
2. *Richiesta dell'Utente:* I dispositivi finali (Talker/Listener) inviano le loro richieste di servizio al CUC tramite protocolli specifici dell'utente (come OPC UA). Il CUC aggrega queste richieste e le inoltra al CNC attraverso l'interfaccia UNI
3. *Configurazione della Rete:* Il CNC riceve le richieste, calcola i percorsi di routing ottimali e le schedulazioni temporali complesse (come le Gate Con-

troil List per lo standard 802.1Qbv). Converte poi questi calcoli in "profili" YANG e li distribuisce agli switch fisici tramite NETCONF. Infine, conferma l'avvenuta allocazione al CUC

4. *Monitoraggio e Ripristino*: La rete viene costantemente monitorata. Se si verifica un guasto, i nodi adiacenti lo segnalano al CNC, che ricalcola istantaneamente le rotte e invia nuove configurazioni per isolare o aggirare il problema

1.5 Wi-Fi 7

Obiettivo di questa sezione è descrivere le funzionalità introdotte in 802.11be e come esse risultano fondamentali per permettere lo sviluppo di infrastrutture di comunicazioni deterministiche su reti wireless, per un introduzione allo standard 802.11 si veda [26]. Le caratteristiche principali introdotte in 802.11be sono:

- **Banda a 320 MHz**
- **MRU - Multiple Resource Unit**: A partire da 802.11ax - Wifi 6 viene utilizzato OFDMA per dividere un canale in diverse RU - *Resource Unit*, ovvero un blocco di sottoportanti assegnato ad una data stazione, in 802.11be viene introdotta la possibilità di assegnare ad una stessa stazione per uno stesso PPDU - *Physical Layer Protocol Data Unit* più RU. Oltre ad incrementare il throughput e l'efficienza ciò permette di effettuare il *preamble puncturing*, ovvero indicare in un campo specifico del frame che uno o più canali del RU sono "forati", così facendo il ricevente eviterà di demodulare quella specifica frequenza.

Questo problema nasce dal fatto che alcune porzioni di frequenza 5GHz sono condivise con radar militari/meteorologici. Per maggiori informazioni si veda [3]

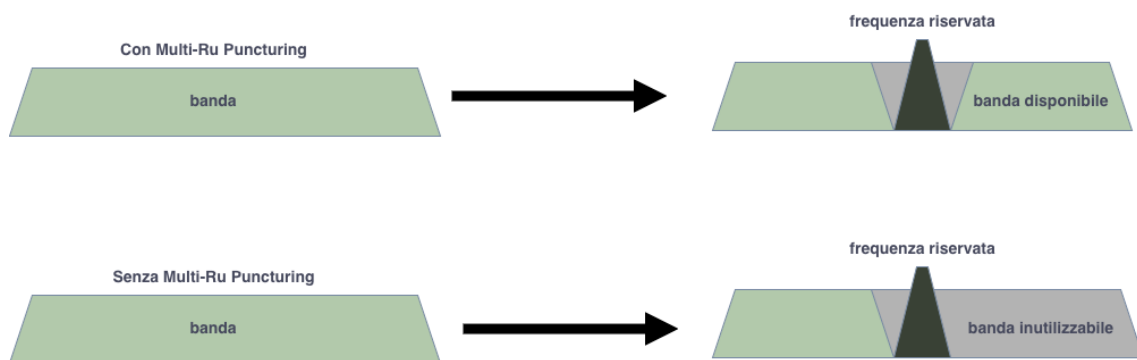


Figura 1.10: Effetti del preamble puncturing nella gestione della banda

In figura 1.10 è mostrata la differenza tra un frame senza *preamble puncturing*, in cui il canale a causa di una frequenza riservata perde la possibilità di utilizzare una porzione di banda disponibile, ed il caso con *preamble puncturing*, dove è possibile osservare come una porzione minima, ed equivalente alla frequenza inutilizzabile del canale, è "forata" e non viene quindi utilizzata per la trasmissione.

- **Modulazione 4096 QAM:** Si ha un miglioramento del 20% rispetto a WiFi 6/6E
- **1024 A-MPDU:** Aggregate MAC Protocol Data Unit introdotto inizialmente nel 802.11n permette di aggregare più MPDU in unico frame, a cui viene assegnato un solo header del livello fisico, questo e il fatto che il ricevente manderà un solo block acknowledgment permette di sfruttare in maniera più efficiente il canale radio e di diminuire il tempo di contesa del canale, 802.11be permette di aggregare fino a 1024 MPDU
- **MLO - Multi-Link Operation:** Permette ad una stazione di utilizzare più interfacce radio contemporaneamente, anche su bande diverse, per trasmettere e ricevere dati. Ciò consente di aumentare la capacità complessiva, ridurre la latenza e migliorare l'affidabilità, poiché se un link è congestionato o ha interferenze, la stazione può continuare a comunicare attraverso gli altri link disponibili.

Una delle principali innovazioni apportate in 802.11be, permette alle stazioni e agli access-point di trasmettere e ricevere dati appartenenti allo stesso flusso di traffico attraverso diverse interfacce radio. Vengono definiti MLD - *Multi-Link Device* sia gli access point che le stazioni dotati di molteplici interfacce radio, i quali, dopo aver stabilito un'unica connessione possono comunicare contemporaneamente su più bande.

In figura 1.11 è mostrato un esempio di MLO, in cui 2 MLD, una stazione e un access point entrambi dotati di tre interfacce radio, comunicano sui diversi link.

Si ha un unico indirizzo MAC per MLD ma nonostante questo, i link sono indipendenti (ognuno gestisce il proprio CSMA/CA, back-off e NAV). Le modalità di utilizzo possibile definite dallo standard sono

1. **STR - Simultaneous Transmit e Receive:** Il dispositivo MLD trasmette e riceve in parallelo su link diversi, massima efficienza ma costo computazionale e consumi elevati
2. **NSTR - Non-Simultaneous Transmit e Receive:** Se i link sono troppo vicini la trasmissione e la ricezione contemporanea potrebbe creare

delle interferenze in-device, in questo modalità non è possibile inviare e ricevere contemporaneamente, ma solo inviare o ricevere.

3. **eMLSR - Enhanced Multi-Link Single Radio:** Introdotta per dispositivi a basso consumo, utilizza un solo canale radio e si sposta dinamicamente tra i link a seconda del traffico. [2]

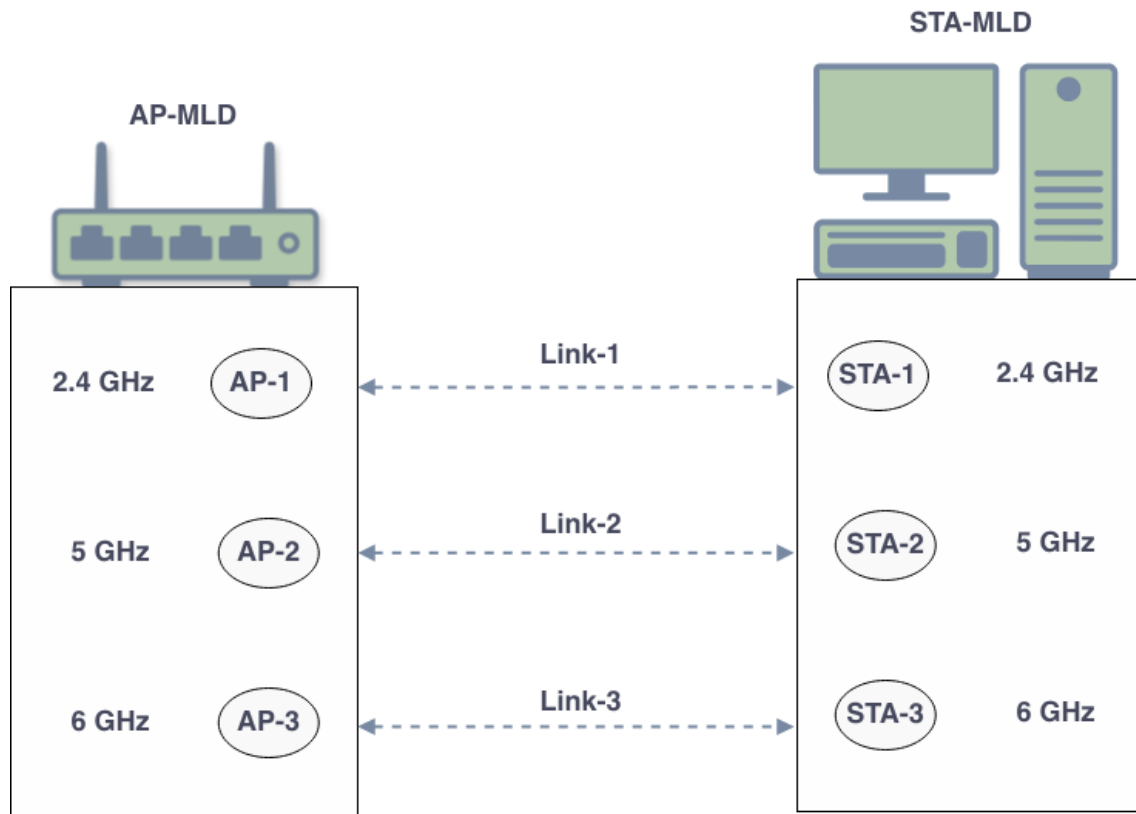


Figura 1.11: Diagramma MLO

- **R-TWT - Restricted Target Wake Time:** È l'evoluzione dei meccanismi di risparmio energetico nei dispositivi wireless, PSM e TWT, introdotto in 802.11be estende la possibilità di negoziare i SP - *Service Period* e WI - *Wake Interval* già presente in TWT, supportando accesso esclusivo al canale di comunicazione durante il SP a determinate STA o TID - *Traffic Identifier*. Questo meccanismo implica quindi una determinata QoS e si avvicina a ciò che abbiamo già visto con lo standard 802.1Qbv, per più dettagli su R-TWT si veda [16]

1.5.1 Supporto al TSN

Le reti wireless hanno storicamente operato secondo il paradigma *best-effort*: il mezzo condiviso, l'accesso probabilistico CSMA/CA e il fading radio rendevano impossibile garantire latenza bounded o affidabilità deterministica. Questa limitazione ha confinato Wi-Fi a scenari *consumer*, escludendolo dagli ambiti industriali, medicali e di controllo in tempo reale, dove IEEE 802.1 TSN stava invece diventando lo standard di riferimento per le reti cablate.

Wi-Fi 7 nasce esplicitamente per rispondere a questo obiettivo: comunicazioni EHT - *Extremely High Throughput* combinate con latenza deterministica e ultra-reliability, colmando il gap tra Wi-Fi e TSN.

È possibile trovare lavori in letteratura [1], dove viene sottolineata la possibilità di mappare il protocollo 802.1CB su 802.11be grazie all'introduzione del MLO.

Capitolo 2

Stack 802.11 in Linux e supporto al TSN

2.1 Stack 802.11

Originariamente, la gestione del wireless in Linux era affidata alle WEXT - *Wireless Extensions*. Sebbene funzionali per i primi standard (802.11b/g), le WEXT presentavano limitazioni strutturali, come l'uso di `ioctl` e l'incapacità di scalare verso configurazioni multi-interfaccia complesse. Ad oggi per fornire un canale di comunicazione bidirezionale tra lo *userspace* e il *kernel space* sono disponibili le Netlink socket, un meccanismo nato come estensione delle socket standard per supportare IPC.

Per un approfondimento sul perchè il passaggio alle socket Netlink sia stato necessario si veda [17].

L'architettura moderna mostrata in figura 2.1, che andremo a considerare d'ora in poi, si basa su una struttura composta da molteplici moduli kernel i quali implementano le due seguenti funzionalità:

- **SME - Station Management Entity:** È l'entità logica di gestione di più alto livello all'interno di una stazione 802.11. Opera al di sopra del MAC layer e ha la responsabilità di prendere decisioni di policy: quali reti scansionare, quando associarsi, quando rieseguire l'autenticazione, quando cambiare canale
- **MLME - MAC Layer Management Entity:** È l'entità di gestione del sottostrato MAC. Se la SME decide cosa fare, la MLME esegue tali decisioni a livello MAC, generando e interpretando i frame di gestione 802.11 (Probe Request/Response, Authentication, Association, Deauthentication, ecc.).

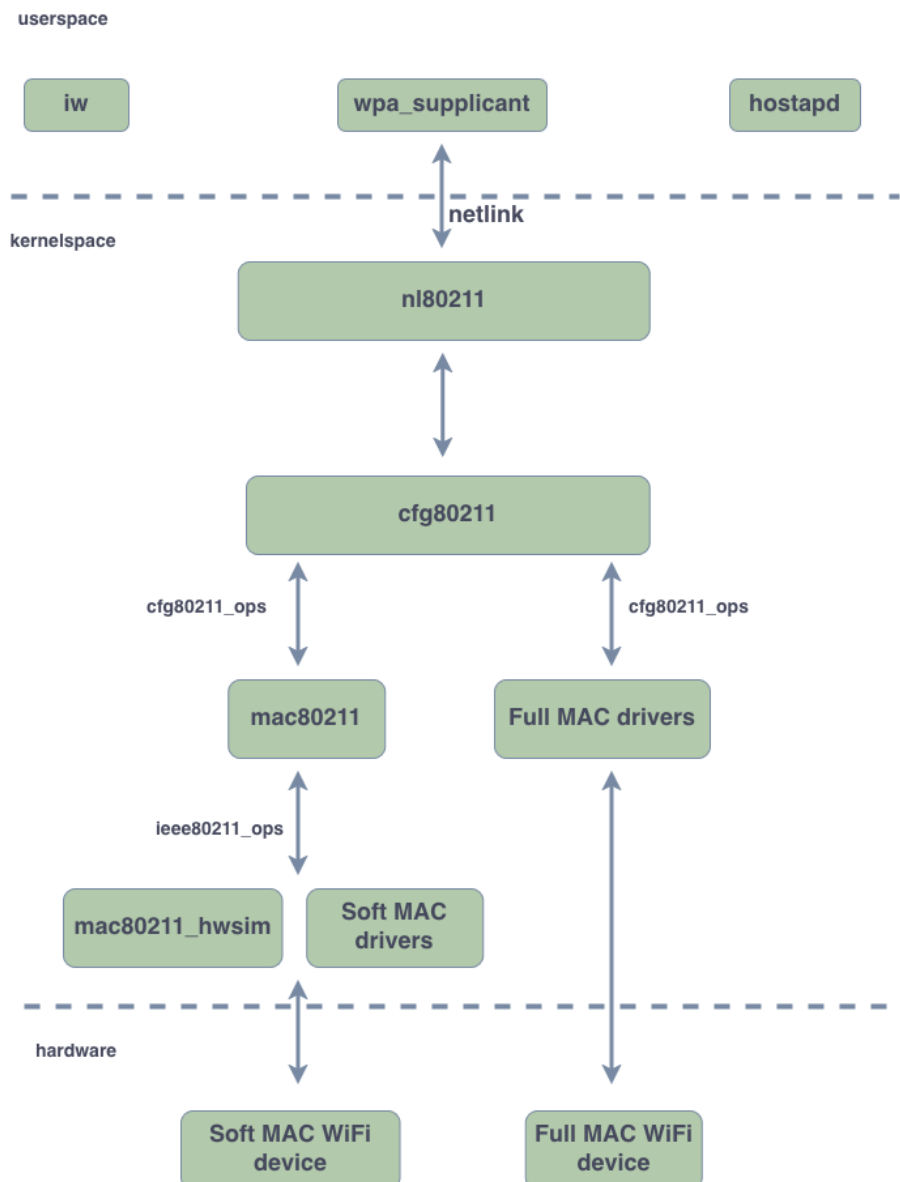


Figura 2.1: Stack 802.11 in Linux

I principali moduli kernel che implementano queste funzionalità sono:

- **nl80211:** Costituisce l'interfaccia standard di comunicazione tra le applicazioni in userspace e il sottosistema wireless del kernel. Attraverso di essa, diversi strumenti software possono interrogare, configurare e controllare le interfacce IEEE 802.11 in modo uniforme, senza dipendere direttamente dai dettagli implementativi del driver sottostante. Gli utenti di `nl80211` sono:

- **iw**: Utility di configurazione a basso livello che consente di eseguire operazioni puntuali quali la scansione delle reti disponibili, la lettura delle capacità del dispositivo e la modifica dei parametri dell'interfaccia
 - **wpa_supplicant e hostapd**: Impiegati rispettivamente per la gestione di una stazione client e di un access point
 - **crda**: Dedicato alla gestione del dominio regolatorio
 - **iwd**: Demone per la gestione delle connessioni Wi-Fi
 - **NetworkManager**: Realizza alcune funzioni di discovery
- **cfg80211**: Rappresenta il sottosistema di configurazione centrale per 802.11 in Linux fornendo un'astrazione unificata dei dispositivi wireless: wiphy
 - **mac80211**: È il framework di gestione del livello MAC per i dispositivi definiti SoftMAC.
 In un'architettura SoftMAC il dispositivo fisico si occupa solamente del livello fisico, mentre la gestione del protocollo è demandata al processore tramite appunto il modulo kernel mac80211
 Invece in un'architettura FullMAC, i dispositivi integrano la logica MLME direttamente nel firmware del dispositivo. La scheda di rete appare al sistema operativo come un'entità autonoma che gestisce internamente la complessità del protocollo 802.11.
 - **mac80211_hwsim**: Nel contesto dello stack wireless Linux, mac80211_hwsim è un modulo del kernel che consente di emulare una o più radio IEEE 802.11 interamente mediante software, senza richiedere hardware fisico dedicato. Dal punto di vista architetturale, mac80211_hwsim sostituisce il livello radio fisico con un meccanismo software che simula la trasmissione dei frame tra interfacce virtuali, mantenendo invariati i livelli superiori dello stack, in particolare mac80211, cfg80211 e nl80211.
 Di conseguenza, applicazioni come iw, wpa_supplicant e hostapd possono essere impiegate senza modifiche anche in ambienti emulati, poiché l'interazione con il sottosistema wireless del kernel avviene secondo le stesse primitive previste per l'hardware reale.
 Questa proprietà rende mac80211_hwsim particolarmente utile in attività di sviluppo, testing e ricerca, dove è importante disporre di scenari wireless controllati, riproducibili e facilmente osservabili.

2.2 Mappare servizi TSN in Linux

Obiettivo di questa sezione è quello di, partendo dagli obiettivi principali del TSN visti in maniera approfondita nel capitolo 1 capire come essi possono essere garantiti in Linux.

2.2.1 Sincronizzazione temporale: Linux PTP

Linux PTP - *Precision Time Protocol* è una suite di software nata per implementare lo standard IEEE 1588, su cui è a sua volta basato lo standard 802.1AS. Fornisce due strumenti principali:

- **ptp4l**: È il demone che implementa a tutti gli effetti il protocollo PTP ed è responsabile della coordinazione e sincronizzazione temporale tra dispositivi
- **phc2sys**: È il demone che sincronizza il PHC, ovvero l'orologio hardware integrato nelle NIC IEEE 1588 compliant, con l'orologio del sistema al fine di garantire consistenza

Gerarchie

Linux PTP sfrutta una struttura gerarchica per delineare diversi tipi di dispositivi all'interno del dominio PTP:

- **GM - Grandmaster Clock**: È l'orologio di riferimento, scelto mediante algoritmo BMCA - *Best Master Clock Algorithm*, rappresenta la sorgente temporale di riferimento per il dominio PTP
- **OC - Ordinary Clock**: È un qualsiasi dispositivo dotato di una singola porta, che si comporta o da master o da slave
- **BC - Boundary Clock**: È un dispositivo dotato di più porte, su una porta, esso si sincronizza rispetto ad un master, e sulle altre porte assume il ruolo di master alla quale degli slave si sincronizzeranno
- **TC - Transparent Clock**: È un dispositivo che ha solo il compito di inoltrare un messaggio PTP, ha il compito di compensare il tempo in cui il messaggio ha passato sul dispositivo di inoltro

Modalità PTP e Scambio di messaggi

Linux PTP supporta due modalità di sincronizzazione:

1. **One-step mode:** I timestamp sono inclusi nei messaggi di Sync
2. **Two-step mode:** I timestamp sono inviati separatamente in dei messaggi di Follow_Up per una maggiore accuratezza

Il processo di sincronizzazione, mostrato in figura 2.2, prevede cinque tipi principali di messaggi:

1. **Announce Message:** Inviato da OC e BC ha lo scopo di pubblicizzare le caratteristiche del proprio orologio e partecipare al processo di selezione del BCMA
2. **Sync Message:** Inviato dal GM trasmette il tempo attuale agli slave per avviare sincronizzazione
3. **Follow_Up:** Inviato dal GM agli slave in modalità two-step, contiene il timestamp di invio del messaggio Sync
4. **Delay_Req:** Inviato dagli slave al GM per misurare il ritardo di propagazione
5. **Delay_Resp:** Inviato dal GM agli slave in risposta a Delay_Req, contiene il timestamp di ricezione del messaggio Delay_Req

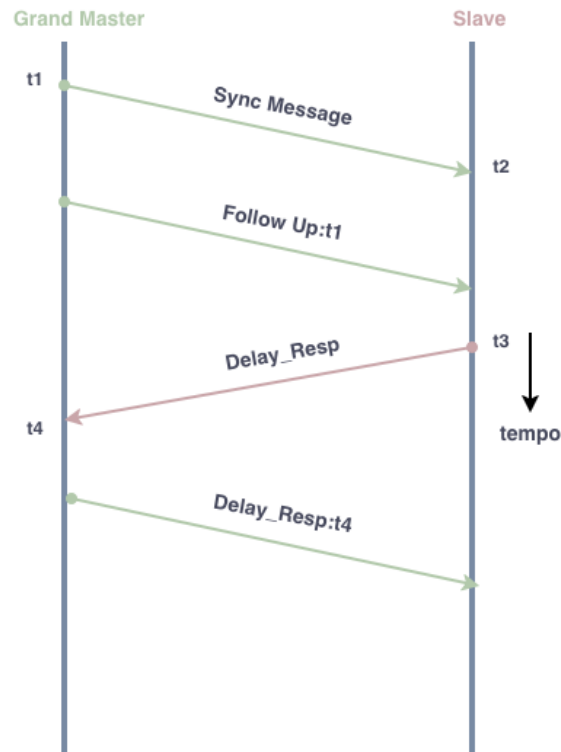


Figura 2.2: Processo di sincronizzazione two-step mode

In figura 2.2 è mostrato un esempio di processo di sincronizzazione in modalità two-step, in cui il GM invia un messaggio Sync agli slave, successivamente invia un messaggio Follow_Up contenente il timestamp di invio del messaggio Sync, gli slave a questo punto possono calcolare la differenza tra il tempo del GM e il proprio tempo locale, e inviare un messaggio Delay_Req al GM per misurare il ritardo di propagazione, infine il GM risponde con un messaggio Delay_Resp contenente il timestamp di ricezione del messaggio Delay_Req, permettendo agli slave di compensare il ritardo di propagazione e sincronizzarsi con il GM. Per informazioni più dettagliate e istruzioni sull'installazione di Linux PTP si veda [20].

Esempio di configurazione e utilizzo Linux PTP

Per mostrare come è possibile utilizzare e configurare Linux PTP verranno utilizzati tre container Docker, node1 e node2 con il ruolo di OC e node3 con il ruolo di GM, in appendice A è riportato lo script bash utilizzato per la configurazione.

Le tre macchine virtuali sono collegate tra loro tramite un bridge Docker, e sono dotate di una scheda di rete virtuale che supporta PTP solamente tramite timestamping software con conseguenze evidenti sulle prestazioni.

Configuriamo le due macchine node1 e node2, che svolgeranno il ruolo di OC, con il seguente file di configurazione per ptp4l

```

1  [global]
2  priority1          128 # Primo criterio di elezione del
   master
3  priority2          128 # Secondo criterio di elezione del
   master per tie-break
4  clockClass         135 # Descrive la qualita della sorgente
   di tempo del clock
5  clockAccuracy       0xFE # Descrive la precisione del clock
   (0xFE indica sconosciuta)
6  offsetScaledLogVariance 0xFFFF # Descrive la varianza scalata
   del clock (0xFFFF indica sconosciuta)
7  domainNumber       0 # Numero di dominio PTP (0-255)
8  logAnnounceInterval 1 # Intervallo dei messaggi di annuncio
9  logSyncInterval    0 # Intervallo dei messaggi di
   sincronizzazione
10 logMinDelayReqInterval 0 # Intervallo minimo di richiesta di
   ritardo
11 announceReceiptTimeout 3 # Intervallo prima che la porta
   dichiari il master assente
12 dataset_comparison  ieee1588 # Algoritmo usato per il
   confronto nel BMCA

```

Listing 2.1: Esempio di configurazione PTP per OC (/etc/ptp4l.conf)

Configuriamo invece node3 con il ruolo di GM nel seguente modo:

```

1  [global]
2  priority1          127
3  priority2          128
4  clockClass         135
5  clockAccuracy       0xFE
6  offsetScaledLogVariance 0xFFFF
7  domainNumber       0
8  logAnnounceInterval 1
9  logSyncInterval    0
10 logMinDelayReqInterval 0
11 announceReceiptTimeout 3
12 dataset_comparison  ieee1588

```

Listing 2.2: Esempio di configurazione PTP per GM (/etc/ptp4l.conf)

Possiamo notare che la configurazione è molto simile, l'unica differenza è il valore di `priority1`, che è più basso per il GM, in questo modo, durante il processo di elezione del master, `node3` sarà preferito rispetto a `node1` e `node2`.

Una volta scritti i file di configurazione, è possibile avviare il servizio eseguendo il comando `ptp4l -i eth0 -f /etc/ptp4l.conf -S -m -q` dove:

- `-i eth0`: Specifica l'interfaccia di rete da utilizzare per la sincronizzazione
- `-f /etc/ptp4l.conf`: Specifica il file di configurazione da utilizzare
- `-S`: Abilita la modalità di timestamping software
- `-m`: Abilita la stampa dei messaggi su stdout
- `-q`: Disabilita logging su syslog

Osservando l'output di `ptp4l` su `node3` nel listato 2.3 è possibile notare che esso assume il ruolo di GM, mentre `node1` e `node2` assumono il ruolo di slave e avviano il processo di sincronizzazione come è possibile osservare nel listato 2.4 (viene mostrato un singolo output in quanto analoghi).

```
1 3fb1aacceb52:/# ptp4l -i eth0 -f /etc/ptp4l.conf -S -m -q
2 ptp4l[5314.111]: port 1 (eth0): INITIALIZING to LISTENING on
  INIT_COMPLETE
3 ptp4l[5314.114]: port 0 (/var/run/ptp4l): INITIALIZING to
  LISTENING on INIT_COMPLETE
4 ptp4l[5314.114]: port 0 (/var/run/ptp4lro): INITIALIZING to
  LISTENING on INIT_COMPLETE
5 ptp4l[5320.303]: port 1 (eth0): new foreign master
  c6b50d.ffff.d8e020-1
6 ptp4l[5320.397]: port 1 (eth0): new foreign master
  3aa0ac.ffff.2a0af2-1
7 ptp4l[5321.213]: port 1 (eth0): LISTENING to MASTER on
  ANNOUNCE_RECEIPT_TIMEOUT_EXPIRES
8 ptp4l[5321.213]: selected local clock a6945b.ffff.0a2fb2 as best
  master
9 ptp4l[5321.213]: port 1 (eth0): assuming the grand master role
```

Listing 2.3: Stdout node3 - processo di elezione del master

```
1 c5a775638645:/# ptp4l -i eth0 -f /etc/ptp4l.conf -S -m -q
2 ptp4l[5312.811]: port 1 (eth0): INITIALIZING to LISTENING on
  INIT_COMPLETE
```

```

3  ptp4l[5312.813]: port 0 (/var/run/ptp4l): INITIALIZING to
LISTENING on INIT_COMPLETE
4  ptp4l[5312.813]: port 0 (/var/run/ptp4lro): INITIALIZING to
LISTENING on INIT_COMPLETE
5  ptp4l[5320.298]: port 1 (eth0): LISTENING to MASTER on
ANNOUNCE_RECEIPT_TIMEOUT_EXPIRES
6  ptp4l[5320.299]: selected local clock c6b50d.ffff.d8e020 as best
master
7  ptp4l[5320.299]: port 1 (eth0): assuming the slave role
8  ptp4l[5324.408]: port 1 (eth0): MASTER to UNCALIBRATED on
RS_SLAVE
9  ptp4l[5325.222]: selected best master clock a6945b.ffff.0a2fb2
10 ptp4l[5325.222]: foreign master not using PTP timescale
11 ptp4l[5327.237]: master offset      -44063 s0 freq   -1563 path
delay      90896
12 ptp4l[5328.238]: master offset      12729 s0 freq   -1563 path
delay      90896
13 ptp4l[5329.239]: master offset     -56051 s0 freq   -1563 path
delay      88968
14 ptp4l[5330.244]: master offset      19991 s0 freq   -1563 path
delay      67801
15 ptp4l[5343.277]: master offset     -35019 s1 freq   -1000 path
delay      54603
16 ptp4l[5344.278]: clockcheck: clock frequency changed
unexpectedly!
17 ptp4l[5344.278]: master offset       9105 s2 freq    -80 path
delay      54603
18 ptp4l[5344.278]: port 1 (eth0): UNCALIBRATED to SLAVE on
MASTER_CLOCK_SELECTED
19 ptp4l[5345.280]: master offset     -37187 s2 freq   -4746 path
delay      54062
20 ptp4l[5346.283]: clockcheck: clock frequency changed
unexpectedly!

```

Listing 2.4: Stdout node1 - processo di elezione del master e sincronizzazione

Nell'output di node1 mostrato nel listato 2.4 è possibile osservare che esso assume il ruolo di slave, e poi la successione di tre fasi principali del processo di sincronizzazione:

- **s0:** Fase iniziale in cui lo *slave* ha appena iniziato a ricevere messaggi dal *master*, ma non ha ancora abbastanza informazioni per calcolare un offset accurato quindi non effettua operazioni di correzione del clock
- **s1:** Fase in cui ptp4l invoca una primitiva di sistema per spostare il clock locale esattamente al tempo del *master*, in questo modo si ha un salto temporale che può essere significativo, ma permette di avvicinare lo slave al

master, non avviene sempre, ma solo quando l'offset è superiore ad una certa soglia chiamata *first_step_threshold*

- **s2:** Fase in cui il protocollo va a regime e lo *slave* corregge continuamente il proprio clock

2.3 Latenza Limitata e 802.1Q: VLAN, TAPRIO, Frame Preemption

Come analizzato nel Capitolo 1, i protocolli e i meccanismi per garantire una latenza limitata e azzerare le perdite causate dalla congestione sono diversi, ma tutti basati sul protocollo 802.1Q, Bridges e Bridges Networks, dove abbiamo visto è centrale il ruolo delle VLAN.

In Linux `iproute2` è un insieme di strumenti a riga di comando per la gestione del networking, basato su Netlink socket per l'interazione con lo stack di rete del kernel, tra le sue diverse funzionalità è presente il supporto alla creazione e gestione di VLAN, per maggiori informazioni si veda [11] e [25].

2.3.1 tc

Tra le diverse funzionalità di `iproute2` è presente anche `tc` - *Traffic Control*, che ci permette di configurare lo scheduler dei pacchetti kernel, i pilastri di `tc` sono:

- **qdisc - Queueing Discipline:** È la disciplina di accodamento, ovvero l'algoritmo che decide quale pacchetto trasmettere, è possibile configurare più `qdisc` in cascata, ad esempio, è possibile utilizzare un `qdisc` per implementare una coda per ogni classe di traffico e un secondo `qdisc` per implementare l'algoritmo di accodamento
- **class:** Rappresenta una classe di traffico, è possibile configurare più classi di traffico, ad esempio, una per il traffico critico e una per il traffico *best-effort*
- **filter:** Permette di classificare i pacchetti in ingresso, ad esempio, è possibile configurare un filtro per classificare i pacchetti in base al campo PCP del frame 802.1Q

Ad oggi i `qdisc` offerti da Linux che supportano TSN sono:

- **taprio:** Implementa l'algoritmo TAS, permette di configurare le GCL per ogni classe di traffico

- **cbs**: Implementa l'algoritmo CBS, permette di configurare i parametri di credito per ogni classe di traffico
- **etf**: Implementa il meccanismo di invio basato sul tempo (TxTime), permettendo di trasmettere i pacchetti a intervalli temporali precisi e predefiniti, garantendo il determinismo nei sistemi real-time

Per maggiori informazioni riguardo gli algoritmi e le loro implementazioni si faccia riferimento alla documentazione ufficiale [10].

Esempio di configurazione taprio tramite tc

In figura 2.5 è riportato un comando per configurare tramite tc una qdisc che implementa taprio:

```

1  tc qdisc add dev eth0 \
2      parent root handle 100: \
3      taprio \
4      num_tc 3 \
5      map 2 2 2 2 1 1 0 0 \
6      queues 1@0 1@1 1@2 \
7      base-time 10000000000 \
8      sched-entry S 01 2000000 \
9      sched-entry S 02 3000000 \
10     sched-entry S 04 5000000 \
11     flags 0x0 \
12     clockid CLOCK_TAI

```

Listing 2.5: Esempio di configurazione di taprio con tc

I campi principali sono:

- **num_tc**: Numero di classi di traffico, in questo caso 3, una per il traffico critico, una per il traffico *time sensitive* e una per il traffico *best-effort*
- **map**: Mappa le classi di traffico appena definite ad una priority (0-7), in questo caso la classe 0 è la più prioritaria associata ai livelli 6 e 7, seguita dalla classe 1 associata ai livelli 4 e 5, e infine dalla classe 2 associata ai livelli 0 e 3. Successivamente la priorità verrà estratta dal campo PCP del frame 802.1Q per classificare i pacchetti in ingresso, o dall'opzione SO_PRIORITY della socket.
- **queues**: Associa le code hardware alle code tc seguendo il formato `numero_code@indice_coda`, in questo caso abbiamo 3 code hardware associate alle 3 classi di traffico, potremmo però avere più code hardware, il che ci permetterebbe di associarne di più alla classe di traffico critica

- **sched-entry**: Ogni riga definisce uno slot della GCL nel formato `sched-entry S bitmap duration`, dove `S` indica che si tratta di uno slot statico, `bitmap` è una bitmap che indica quali classi di traffico possono essere trasmesse durante lo slot, e `duration` è la durata dello slot in nanosecondi
- **flags**: Modalità operativa di taprio, `0x0` indica che taprio è in modalità software, dove i gate sono gestiti dal kernel

2.3.2 Frame Preemption

In Linux, il supporto alla Frame Preemption è fornito attraverso `ethtool` e le `qdisc mqprio` o `taprio`.

`ethtool` è l'utility di amministrazione delle interfacce Ethernet e consente di interrogare e configurare alcune funzionalità avanzate del controller di rete, comprese quelle necessarie per abilitare la preemption. Il meccanismo implementa quanto previsto dagli standard IEEE 802.1Qbu e IEEE 802.3br, distinguendo tra traffico critico, che non può essere interrotto, e traffico preemptable, che può essere sospeso temporaneamente per consentire la trasmissione immediata di frame più urgenti.

Esempio di configurazione frame preemption tramite tc

Per configurare e utilizzare Frame Preemption in Linux è necessario innanzitutto abilitare la funzionalità tramite il comando

```
1  ethtool --set-mm eth0 \
2      pmac-enabled on \
3      tx-enabled on \
4      verify-enabled on \
5      tx-min-frag-size 60
```

Listing 2.6: Abilitazione del MAC Merge layer con `ethtool`

Successivamente sarà necessario configurare una `qdisc` che supporti la preemption, ad esempio `mqprio` o `taprio`, in questo caso mostreremo un esempio di configurazione con `taprio` in quanto già introdotta in precedenza.

```
1  tc qdisc replace dev eth0 \
2      parent root handle 100: taprio \
3      num_tc 3 \
4      map 2 2 2 2 1 1 0 0 \
5      queues 1@0 1@1 1@2 \
6      fp P E P \
7      base-time 10000000000 \
8      sched-entry S 01 2000000 \
```

```

9      sched-entry S 02 3000000 \
10     sched-entry S 04 5000000 \
11     flags 0x2 \
12     clockid CLOCK_TAI

```

Listing 2.7: Configurazione di taprio con Frame Preemption

La configurazione è analoga a quella mostrata nel listato 2.5; le differenze sono il campo `flags` impostato a `0x2`, che pone `taprio` in modalità full offload — i gate e il MAC Merge layer sono gestiti direttamente dalla NIC, condizione necessaria poiché la frammentazione dei frame è una funzione hardware — e l'opzione `fp`, che assegna a ciascuna traffic class il ruolo di preemptable (P) o express (E). Nell'esempio le classi TC0 e TC2 sono prelazionabili e TC1 è express.

In 1, si è discusso di come algoritmi di schedulazione del traffico basati su un clock condiviso non siano sempre ideali, per questo veniva introdotto ATS, attualmente non è presente un'implementazione ufficiale dello standard IEEE 802.1Qcr, ma in letteratura sono presenti lavori [21] che dimostrano che con `tc` si può approssimare ATS nella variante UBS/TBE, nonostante ciò viene sottolineato che per un'implementazione completa servirebbero nuove primitive o nuove `qdisc` dedicate nel kernel.

2.4 Affidabilità: FRER

Nel kernel Linux non è attualmente disponibile un'implementazione nativa e completa dello standard IEEE 802.1CB - Frame Replication and Elimination for Reliability comparabile, ad esempio, al supporto offerto per altri meccanismi TSN tramite `qdisc` dedicate.

Tuttavia, il sistema mette a disposizione strumenti programmabili a basso livello che consentono di realizzarne implementazioni sperimentali. Un esempio rilevante è XDP FRER, che propone una realizzazione parziale del paradigma FRER mediante eBPF/XDP [5].

In tale approccio, la replicazione dei frame viene effettuata su più interfacce di uscita, mentre sul nodo ricevente una funzione di eliminazione provvede a riconoscere e scartare le copie duplicate, consegnando allo stack superiore un solo pacchetto valido. Sebbene questa soluzione non costituisca un supporto standard integrato nel kernel, essa dimostra che Linux dispone di primitive sufficientemente flessibili per implementare meccanismi di affidabilità per-packet in modo efficiente.

Esempio utilizzo XDP FRER

Per mostrare il funzionamento di XDP FRER, faremo riferimento all'ambiente di test fornito dall'autore di [5], in cui viene proposta un'infrastruttura composta da due nodi, un *Transmitter* ed un *Listener*, collegati tra loro tramite una serie di switch virtuali.

Per configurare gli switch virtuali in entrambe le direzioni per abilitare la replicazione ed eliminazione dei frame, è possibile utilizzare i seguenti comandi:

```
1      # transmitter -> listener
2      nsx xdpfrer -m repl -i aeth0:10 -e enp3s0:55 -e enp6s0:56
3      nsx xdpfrer -m elim -i enp4s0:55 -i enp7s0:56 -e beth0:20
4
5      # listener -> transmitter
6      nsx xdpfrer -m repl -i beth0:20 -e enp4s0:66 -e enp7s0:67
7      nsx xdpfrer -m elim -i enp3s0:66 -i enp6s0:67 -e aeth0:10
```

Listing 2.8: Avvio di XDP FRER per replicazione ed eliminazione dei frame

Dove `nsx xdpfrer -m repl -i aeth0:10 -e enp3s0:55 -e enp6s0:56` significa che i pacchetti con VLAN ID 10 in ingresso sull'interfaccia `aeth0` vengono replicati su `enp3s0` e `enp6s0` con VLAN ID 55 e 56 rispettivamente.

Mentre `nsx xdpfrer -m elim -i enp4s0:55 -i enp7s0:56 -e beth0:20` significa che i pacchetti in ingresso su `enp4s0` con VLAN ID 55 e su `enp7s0` con VLAN ID 56 vengono processati per l'eliminazione dei duplicati e consegnati su `beth0` con VLAN ID 20.

Si noti che `nsx` è l'alias per entrare nel namespace dello switch FRER, in questa implementazione il VLAN ID si comporta stream ID per identificare a quale flusso appartiene un frame e come trattarlo (replicare vs eliminare). Senza VLAN diverse per andata/ritorno, il programma XDP non saprebbe distinguere se un frame che arriva su una determinata interfaccia appartiene allo stream in arrivo dal percorso, o è un frame di ritorno da instradare verso il talker.

Possiamo effettuare il test vero e proprio effettuando un ping come mostrato nel listato 2.9 tra il Transmitter ed il Listener, e osservare che nonostante i frame vengano replicati su due percorsi diversi come si può osservare nel listato 2.11, il Listener riceve correttamente i pacchetti senza duplicati come mostrato nel listato 2.10, dimostrando l'efficacia del meccanismo di eliminazione, che si può osservare nel listato 2.12.

```
1      (veth.env) root:test# tx ping 10.0.0.2 -c 4
2      PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
3      64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.274 ms
4      64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.152 ms
5      64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.197 ms
```

6 64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.144 ms

Listing 2.9: Ping tra Transmitter e Listener con XDP FRER abilitato

```
1 (veth.env) root:test# ip netns exec lx tcpdump -i leth0.20 -n
icmp -v
2 tcpdump: listening on leth0.20, link-type EN10MB
(Ethernet), capture size 262144 bytes
3
4 13:04:01.112843 IP (tos 0x0, ttl 64, id 43201, offset 0,
flags [DF], proto ICMP (1), length 84)
5 10.0.0.1 > 10.0.0.2: ICMP echo request, id 1423, seq
1, length 64
6 13:04:01.113021 IP (tos 0x0, ttl 64, id 43201, offset 0,
flags [DF], proto ICMP (1), length 84)
7 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 1423, seq
1, length 64
8
9 13:04:02.114056 IP (tos 0x0, ttl 64, id 43202, offset 0,
flags [DF], proto ICMP (1), length 84)
10 10.0.0.1 > 10.0.0.2: ICMP echo request, id 1423, seq
2, length 64
11 13:04:02.114198 IP (tos 0x0, ttl 64, id 43202, offset 0,
flags [DF], proto ICMP (1), length 84)
12 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 1423, seq
2, length 64
13
14 13:04:03.115312 IP (tos 0x0, ttl 64, id 43203, offset 0,
flags [DF], proto ICMP (1), length 84)
15 10.0.0.1 > 10.0.0.2: ICMP echo request, id 1423, seq
3, length 64
16 13:04:03.115489 IP (tos 0x0, ttl 64, id 43203, offset 0,
flags [DF], proto ICMP (1), length 84)
17 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 1423, seq
3, length 64
18
19 13:04:04.116741 IP (tos 0x0, ttl 64, id 43204, offset 0,
flags [DF], proto ICMP (1), length 84)
20 10.0.0.1 > 10.0.0.2: ICMP echo request, id 1423, seq
4, length 64
21 13:04:04.116902 IP (tos 0x0, ttl 64, id 43204, offset 0,
flags [DF], proto ICMP (1), length 84)
22 10.0.0.2 > 10.0.0.1: ICMP echo reply, id 1423, seq
4, length 64
23
24 4 packets captured
25 4 packets received by filter
26 0 packets dropped by kernel
```

Listing 2.10: tcpdium su Listener

```

1 (veth.env) root:test# nsx ../src/xdpfrer -m repl -i aeth0:10 -e
  enp3s0:55 -e enp6s0:56
2      Config replication on interface aeth0 (ifindex: 2) match id
  10
3      Received: 1
4      Received: 2
5      Received: 3
6      Received: 4

```

Listing 2.11: Stdout di switch virtuale in modalità replicazione

```

1 (veth.env) root:test# nsx ../src/xdpfrer -m elim -i enp3s0:66 -i
  enp6s0:67 -e aeth0:10
2      Passed: 1, Dropped: 1
3      Passed: 2, Dropped: 2
4      Passed: 3, Dropped: 3
5      Passed: 4, Dropped: 4

```

Listing 2.12: Stdout di switch virtuale in modalità eliminazione

In ambito wireless, una discussione analoga può essere sviluppata con riferimento al Wi-Fi 7, che introduce la Multi-Link Operation. Tale funzionalità consente a un dispositivo di utilizzare simultaneamente più link radio, anche su bande differenti, aumentando robustezza e prestazioni.

In linea teorica, ciò potrebbe permettere la trasmissione ridondata dello stesso pacchetto su più collegamenti radio o catene trasmissive, con finalità simili a quelle perseguite da FRER.

Tuttavia, per parlare propriamente di FRER sarebbe necessario affiancare a questa capacità anche un meccanismo esplicito di numerazione, riconoscimento ed eliminazione dei duplicati, che non è implicito nelle sole funzionalità multi-link del Wi-Fi 7.

2.5 Gestione delle risorse

Nel contesto Linux, i concetti di controllo e configurazione definiti dagli standard TSN si mappano attraverso una chiara separazione tra piano di controllo e piano dati.

Il sistema operativo fornisce infatti i meccanismi locali necessari all'esecuzione delle politiche TSN, come appena mostrato, mentre le decisioni globali di configurazione sono demandate a entità esterne quali CUC e CNC.

In tale prospettiva, gli standard come IEEE 802.1Qcp trovano applicazione mediante modelli YANG e protocolli NETCONF/RESTCONF, usati per trasportare

verso i nodi Linux configurazioni astratte che vengono poi tradotte in regole concrete del sottosistema di rete.

Esempio di utilizzo RESTCONF e modello YANG per configurazione PTP

Nel seguente esempio, mostriamo come è possibile replicare la configurazione di Linux PTP mostrata in 2.2.1 utilizzando un modello YANG e RESTCONF.

Nel listato 2.13 è riportato un esempio di comando `curl` per inviare una richiesta RESTCONF di tipo PUT a `node1`, con l'obiettivo di configurare il servizio PTP secondo i parametri definiti nel listato 2.14 che rappresenta il modello YANG corrispondente al listato 2.1.

```
1  curl -X PUT \  
2      -H "Content-Type:_application/yang-data+json" \  
3      -H "Accept:_application/yang-data+json" \  
4      -u admin:password \  
5      http://node1:8008/restconf/data/ieee1588-ntp-tt:ntp  
6      /instance=0 \  
7      -d @ntp_node1.json
```

Listing 2.13: PUT RESTCONF per configurare Linux PTP su `node1`

```
1  {  
2      "ieee1588-ntp-tt:instance": [  
3          {  
4              "instance-index": 0,  
5  
6              "default-ds": {  
7                  "domain-number": 0,  
8                  "priority1": 128,  
9                  "priority2": 128,  
10                 "dataset-comparison": "ieee1588",  
11  
12                 "clock-quality": {  
13                     "clock-class": 135,  
14                     "clock-accuracy": 254,  
15                     "offset-scaled-log-variance": 65535  
16                 }  
17             },  
18  
19             "port-ds-list": [  
20                 {  
21                     "port-index": 1,  
22                     "underlying-interface": "eth0",  
23  
24                     "log-announce-interval": 1,  
25                     "announce-receipt-timeout": 3,  
26                     "log-sync-interval": 0,
```

```

27         "log-min-delay-req-interval": 0,
28
29         "delay-mechanism": "e2e",
30         "port-enable": true
31     }
32 ]
33 }
34 ]
35 }

```

Listing 2.14: Payload JSON per configurare Linux PTP su node1 (ptp_node1.json)

Si noti che i valori precedentemente definiti nel file di configurazione `ptp41.conf` erano in formato esadecimale, mentre nel payload JSON sono rappresentati in formato decimale, questo perché il modello YANG definisce i campi come interi.

Capitolo 3

Prototipo

L'intero codice sorgente sviluppato per la realizzazione del prototipo, comprendente gli script di configurazione dei network namespace, l'infrastruttura di routing e il codice di test, è accessibile e consultabile pubblicamente. Il repository GitHub contenente i file di progetto e le istruzioni per la riproduzione dell'ambiente emulato è disponibile al seguente indirizzo: <https://github.com/domenicoformosi/TSN-over-WiFi7>.

Per un approfondimento sulla macchina host e l'ambiente utilizzato per lo sviluppo si veda l'appendice B.

3.1 Obiettivi e assunzioni

Il prototipo presentato in questo capitolo si pone come obiettivo quello di mostrare come sia possibile far coesistere le caratteristiche principali del TSN e protocolli di gestione del livello fisico e MAC di ultima generazione come 802.11be.

Gli obiettivi del TSN a cui il prototipo vuole andare in contro sono:

- Sincronizzazione temporale
- Affidabilità
- Latenza limitata

In figura 3.1 è possibile osservare un diagramma in cui sono presenti i principali componenti del prototipo e come essi comunicano.

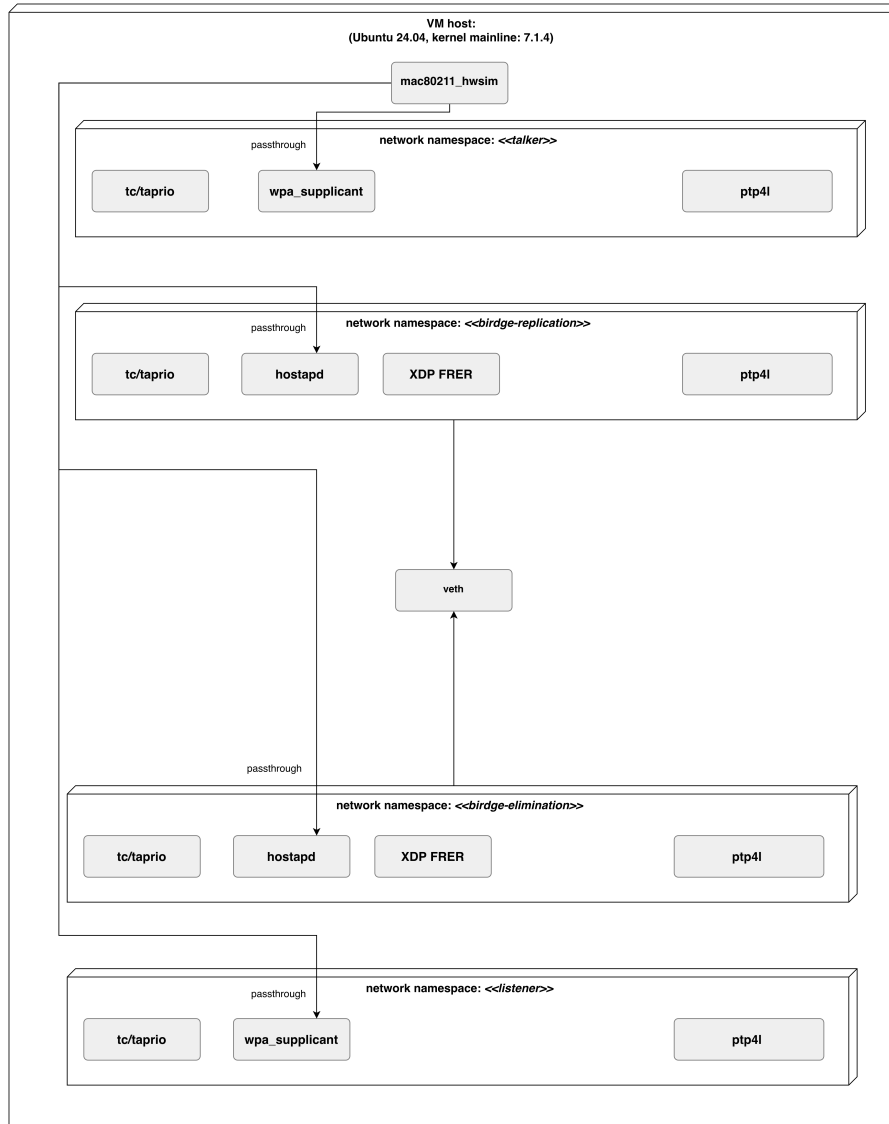


Figura 3.1: Diagramma dei componenti prototipo

L'implementazione del prototipo si basa su un insieme di assunzioni:

- Gli endpoint della comunicazione sono un `talker` ed un `listener`, ci interessa garantire determinate caratteristiche di qualità ed affidabilità in maniera unidirezionale
- Il prototipo sfrutta emulazione di hardware Wi-Fi 7 tramite `mac80211_hwsim`, modulo kernel citato nella sezione 2.1.

Questo approccio permette di dimostrare la fattibilità a livello infrastruttura-

le, mostrando che i vari protocolli possono lavorare insieme, ha però richiesto di accettare dei compromessi in termini di prestazioni e funzionalità

- Le tecniche di isolamento della rete utilizzate vincolano l'intera infrastruttura ad avere un unico kernel ed un unico dominio di clock

Per un'analisi più dettagliata delle conseguenze causate dall'emulazione hardware e dalla presenza di un unico kernel si veda la sezione 3.3.

3.2 Struttura

Il modello utilizzato per il prototipo, in maniera coerente con l'obiettivo principale di voler dimostrare la fattibilità dell'infrastruttura stessa, non prevede entità centrali come CNC o CUC visti in 1.4.2 o 1.4.3, nè meccanismi di configurazione distribuiti visti in 1.4.1.

Nello specifico verranno implementati i protocolli:

- PTP mediante `ptp4l`
- TAS mediante il comando `tc` e la *queueing discipline* `taprio`
- FRER mediante l'implementazione sperimentale XDP FRER [5]

La topologia di rete del prototipo, mostrata in figura 3.2, prevede che i collegamenti mediante Wi-Fi 7 siano quelli tra il `talker` e `bridge-repl`, ed il `listener` ed il `bridge-elim`, i due bridge sono connessi tra loro tramite un `veth D.1`.

Le interfacce di rete Wi-Fi 7 sono emulate tramite il modulo `mac80211_hwsim` e iniettate nei diversi *network namespaces* D.2, sulla quale verranno utilizzati i tool `wpa_supplicant` e `hostapd` per gestire rispettivamente stazioni client - STA e access point - AP. Per un approfondimento sulla configurazione dei due tool si veda l'appendice C.

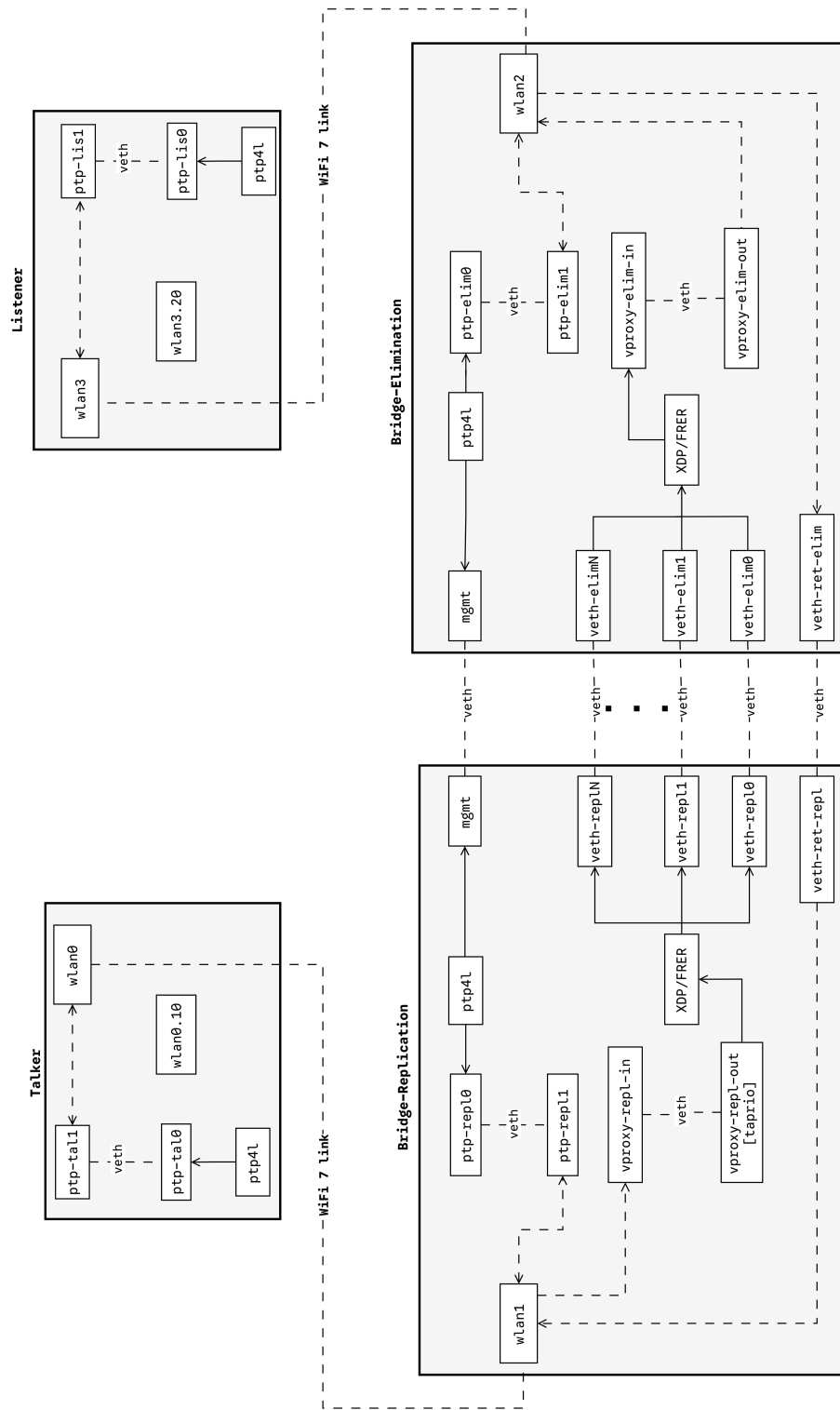


Figura 3.2: Topologia di rete del prototipo

3.3 Motivazioni e scelte progettuali

In fase di progettazione è stata inizialmente considerata l'opzione di utilizzare tecnologie di containerizzazione come LXD/LXC o Podman, ma è stato infine scelto di utilizzare i network namespace offerti dal kernel Linux (per un approfondimento si veda la sezione D.2).

Le motivazioni dietro questa scelta risiedono nel livello di astrazione introdotto dall'utilizzo dei container, il quale rende le operazioni di configurazione dello stack di rete problematiche e, a volte, non realizzabili in questo specifico scenario. I network namespace forniscono uno spazio dei nomi isolato per interfacce di rete, socket e tabelle di routing, permettendo di lavorare in maniera più granulare sullo stack di rete. In entrambi i casi (namespace e container), tutte le entità condividono lo stesso kernel e lo stesso dominio di clock; ciò comporta che la sincronizzazione temporale non sarà valutabile dal punto di vista dell'accuratezza dei risultati, ma assumerà una valenza puramente infrastrutturale per il prototipo. Lo stesso limite concettuale si applica alle tecniche di schedulazione del traffico. Infine, è importante sottolineare i vincoli intrinseci nati dall'emulazione dell'hardware Wi-Fi 7: una NIC Wi-Fi 7 fisica può esporre diverse code hardware sulle quali il kernel andrà a depositare i pacchetti, e sarà poi la NIC stessa a implementare le funzioni del livello MAC e del livello fisico, come ad esempio la *frame preemption*. La presenza di code multiple, come discusso nel capitolo 1, è fondamentale per le tecniche di schedulazione del traffico, poiché permette di suddividerlo in diverse classi gestite secondo logiche deterministiche.

Andando ad emulare un dispositivo Wi-Fi 7, nello specifico utilizzando `mac80211_hwsim`, non si ha a disposizione un hardware virtuale dotato di code multiple, rendendo di fatto impossibile, *senza ricorrere ad accorgimenti ad-hoc*, l'utilizzo di protocolli basati sul *Traffic Control* come `taprio`.

Inoltre, non sarà possibile implementare le tecniche di *frame preemption* viste in 1.2.1, in quanto il dispositivo emulato utilizza dei loopback virtuali per comunicare e non possiede un livello MAC effettivo (non sono disponibili i sottolivelli MAC Merge Layer, pMAC ed eMAC).

Nelle sezioni successive verranno presentate, per ogni protocollo implementato, le problematiche riscontrate in fase di progettazione e le rispettive soluzioni architetturali, evidenziando come alcune di queste operino in sinergia.

3.3.1 FRER

In figura 3.2 è possibile notare che i bridge sono collegati tra loro tramite diverse veth, e non tramite un collegamento radio come avviene invece tra bridge ed endpoint. La necessità di utilizzare un collegamento ethernet, virtuale o meno (rappresentato in figura dalle coppie `veth-repl-0:veth-elim0`, `veth-repl-1:veth-elim1`,

... , veth-repl-N:veth-elimN), nasce dall'utilizzo di un'implementazione sperimentale del protocollo FRER (analizzata in 2.4) che attualmente non supporta la duplicazione e rimozione dei pacchetti mediante collegamenti radio.

In aggiunta, il programma XDP FRER non può elaborare direttamente i pacchetti in ingresso dalle interfacce radio a causa della natura del framework mac80211: quest'ultimo non implementa gli hook `ndo_bpf` e `ndo_xdp_xmit`, i quali sono requisiti fondamentali per poter eseguire XDP in modalità `XDP_MODE_NATIVE`. I pacchetti provenienti dalle interfacce radio costringono quindi all'utilizzo di XDP in modalità `XDP_MODE_SKB`. Come è possibile osservare nel listato 3.1,

```
1  static u32 netif_receive_generic_xdp(struct sk_buff *skb,
2                                     struct xdp_buff *xdp,
3                                     struct bpf_prog *xdp_prog)
4  {
5      ...
6      if (skb_cloned(skb) || skb_is_nonlinear(skb) ||
7          skb_headroom(skb) < XDP_PACKET_HEADROOM) {
8          if (netif_skb_check_for_xdp(skb, xdp_prog))
9              goto do_drop;
10         }
11         ...
12     }
```

Listing 3.1: fonte: <https://github.com/torvalds/linux/blob/master/net/core/dev.c>

se il programma XDP riceve un `sk_buff`¹ non lineare, tenta di ripristinarlo richiamando il metodo `netif_skb_check_for_xdp(skb, xdp_prog)`; in caso di fallimento, il pacchetto subisce un drop.

I dispositivi Wi-Fi, specialmente quelli in grado di aggregare molti MPDU in un unico frame, utilizzano un approccio di accesso alla memoria di tipo *Scatter/Gather*, il quale dà vita nativamente a `sk_buff` non lineari. Durante i test effettuati, si è dimostrato empiricamente che la procedura di ripristino o linearizzazione di questi `sk_buff` fallisce sistematicamente.

Per risolvere questa incompatibilità, l'interfaccia radio viene incapsulata all'interno di una *veth*, la quale ha il compito di linearizzare lo `sk_buff` sfruttando l'azione `redirect` del sottosistema `tc`. Questa soluzione di incapsulamento risolve il limite imposto da XDP e, come si vedrà nella sezione 3.3.2, fornisce contemporaneamente l'infrastruttura necessaria ad aggirare i limiti delle code radio.

Considerando il Bridge Replication presente in figura 3.2, viene creata una coppia *veth* nominata `vproxy-repl-in:vproxy-repl-out` e confinata all'interno del namespace. Successivamente, tramite i comandi mostrati nel listato 3.2, viene aggiunta una *queuing discipline* in ingresso all'interfaccia `wlan1`. Tramite l'a-

¹La struttura dati principale utilizzata nel kernel Linux per rappresentare un pacchetto di rete.

zione `redirect`, i pacchetti che soddisfano le condizioni `flower vlan_id 10 vlan_prio 5` vengono reindirizzati verso `vproxy-repl-in`. Essendo quest'ultima uno degli endpoint della veth, i pacchetti emergono linearizzati sull'interfaccia `vproxy-repl-out`, sulla quale il programma XDP FRER sarà in ascolto per effettuare la replicazione (come mostrato nel listato 3.3).

Si noti che in 3.2 è presente anche l'azione `skbedit priority 5`. Questo accorgimento è strettamente necessario per poter implementare correttamente `taprio` sulla medesima topologia; il suo ruolo tecnico verrà approfondito nella sottosezione successiva.

```
1 ip netns exec bridge-repl tc qdisc add dev wlan1 ingress
2 ip netns exec bridge-repl tc filter add dev wlan1 ingress
  protocol 802.1Q prio 1 flower vlan_id 10 vlan_prio 5 action
  skbedit priority 5 pipe action mirrored egress redirect dev
  vproxy-repl-in
```

Listing 3.2: Comandi per incapsulare wlan1 (scripts/wrapper_tc.sh)

```
1 ip netns exec bridge-repl stdbuf -oL -eL xdpfrer -m repl -i
  vproxy-repl-out:10 -e veth-repl:55 -e veth-repl-b:56
  >/tmp/frer_repl.log 2>&1 &
```

Listing 3.3: Comando per lanciare programma di replicazione XDP (scripts/launch_frer.sh)

3.3.2 TAPRIO

È stato precedentemente sottolineato come, su un dispositivo emulato tramite `mac80211_hwsim`, non sia possibile applicare direttamente la *queuing discipline* `taprio`, a causa della presenza di una singola coda di trasmissione.

Tuttavia, durante la creazione di una veth, il kernel permette di specificare arbitrariamente il numero di code da allocare. La soluzione a questo limite si sovrappone elegantemente a quella adottata per il protocollo FRER: è sufficiente riutilizzare la medesima interfaccia virtuale `vproxy-repl-in:vproxy-repl-out` introdotta per linearizzare gli `sk_buff`, assicurandosi di dichiarare il numero di code necessarie in fase di istanziazione, come evidenziato nel listato 3.4.

```
1 ip netns exec bridge-repl ip link add vproxy-repl-in numtxqueues
  3 numrxqueues 3 type veth peer name vproxy-repl-out numtxqueues 3
  numrxqueues 3
```

Listing 3.4: Creazione veth proxy con 3 code per supportare `taprio` (scripts/namespace_setup.sh)

Una volta ridiretti i pacchetti in ingresso da `wlan1` verso `vproxy-repl-in` (tramite `tc redirect`), subentra la necessità dell'azione `skbedit priority 5`, anticipata nel listato 3.5.

```

1 ip netns exec bridge-repl tc filter add dev wlan1 ingress
   protocol 802.1Q prio 1 flower vlan_id 10 vlan_prio 5 action
   skbedit priority 5 pipe action mirred egress redirect dev
   vproxy-repl-in

```

Listing 3.5: Ridirezione dei pacchetti entranti in wlan1 a vproxy-repl-in (scripts/wrapper_tc.sh)

Tramite l'azione `skbedit`, si va a modificare esplicitamente il campo `priority` della struttura `sk_buff` del kernel, impostandolo a 5. Questa operazione manuale è imprescindibile: il qdisc `taprio` si basa esclusivamente su questo parametro interno (`skb->priority`) per assegnare i pacchetti alle corrette *Traffic Class* (TC). Effettuando una ridirezione diretta (tramite l'azione `mirred`), il pacchetto scavalca i normali percorsi di elaborazione dello stack di rete. In assenza di questo intervento mirato, il valore PCP presente nel tag VLAN originale non verrebbe propagato o mappato automaticamente, privando di fatto `taprio` delle informazioni necessarie per la schedulazione.

Infine, attraverso il comando riportato nel listato 3.6, si associano le 3 classi di traffico alle rispettive code e si configurano le finestre temporali (Gate Control List) per la trasmissione deterministica.

```

1 ip netns exec bridge-repl tc qdisc replace dev vproxy-repl-in
   root taprio \
2     num_tc 3 \
3     map 0 0 0 1 0 2 0 0 0 0 0 0 0 0 0 0 \
4     queues 1@0 1@1 1@2 \
5     base-time 0 \
6     sched-entry S 04 1000000 \
7     sched-entry S 02 8000000 \
8     sched-entry S 01 1000000 \
9     clockid CLOCK_TAI

```

Listing 3.6: Creazione qdisc `taprio` (scripts/launch_taprio.sh)

3.3.3 PTP

In assenza di hardware dotato di *Physical Hardware Clock* - PHC e capacità di *hardware timestamping*, l'utilizzo di `ptp4l` richiede che l'interfaccia di rete sia dotata delle *capabilities* `software-receive` e `software-transmit`.

I dispositivi creati mediante `mac80211_hwsim` dispongono unicamente della *capability* `software-receive`. Tuttavia, come nei casi precedenti, le interfacce `veth` possiedono entrambe le *capabilities*: risulta quindi sufficiente incapsulare l'interfaccia radio all'interno di una `veth` per consentirle di comunicare correttamente con il processo `ptp4l`.

Nella figura 3.3 è possibile osservare lo schema dell'incapsulamento all'interno del Talker. In modo analogo ai casi precedenti, l'unica differenza risiede nella bidirezionalità della comunicazione: se prima il traffico procedeva dal Talker verso il Listener tramite il percorso FRER e tornava indietro su un percorso dedicato, in questo caso il traffico deve poter viaggiare dal processo ptp4l all'interfaccia wlan0 e viceversa.

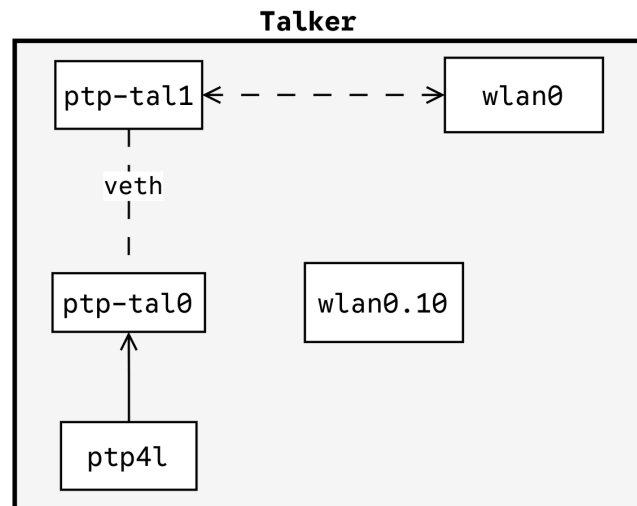


Figura 3.3: Incapsulamento wlan0 per l'utilizzo di ptp4l

Supponendo di aver attivato il processo ptp4l sull'interfaccia ptp-tal0, risulta necessario definire due regole distinte mediante `tc redirect`. La prima, mostrata nel listato 3.7, si occupa di reindirizzare tutto il traffico generato da ptp4l verso wlan0.

```
1 ip netns exec talker tc filter add dev ptp-tal1 parent ffff:
  protocol all u32 match u32 0 0 at 0 action mirred egress redirect
  dev wlan0
```

Listing 3.7: Redirect dal processo ptp4l a wlan0 (scripts/wrapper_tc.sh)

La seconda regola, mostrata in 3.8, effettua invece il redirect su ptp-tal1 esclusivamente del traffico appartenente al protocollo PTP (EtherType 0x88f7) in ingresso su wlan0.

```
1 ip netns exec talker tc filter add dev wlan0 parent ffff:
  protocol 0x88f7 prio 1 u32 match u32 0 0 at 0 action mirred
  egress redirect dev ptp-tal1
```

Listing 3.8: Redirect da wlan0 al processo ptp4l (scripts/wrapper_tc.sh)

Il ragionamento applicato in questo caso al Talker rimane valido per ogni nodo del prototipo. Tuttavia, sui Bridge, il processo `ptp4l` deve essere in ascolto su due interfacce distinte: quella che lo collega alla sua controparte e quella che lo collega all'endpoint.

Considerando il caso del Bridge Replication mostrato in figura 3.4, il processo `ptp4l` è attivo, tramite incapsulamento, sull'interfaccia `wlan1` (che lo connette attraverso Wi-Fi 7 al Talker) e sull'interfaccia `mgmt`, la quale rappresenta uno dei due endpoint della veth che collega il namespace Bridge Replication a quello Bridge Elimination.

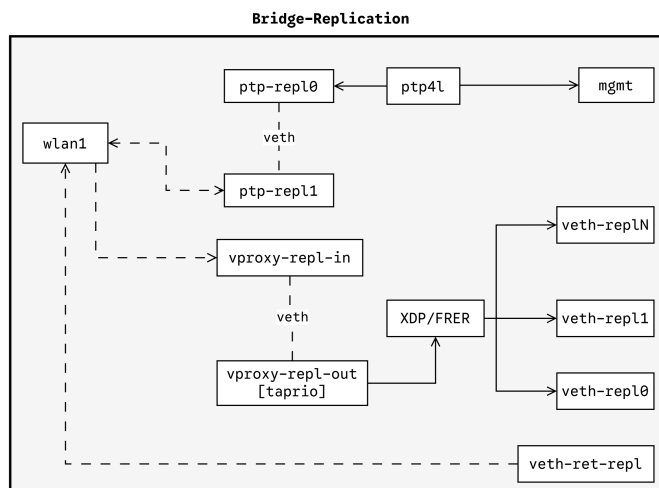


Figura 3.4: Incapsulamento wlan1 per l'utilizzo di ptp4l

La necessità di utilizzare una veth dedicata esclusivamente allo scambio di traffico PTP tra i due Bridge deriva dal comportamento del programma XDP FRER, il quale richiede che l'intero traffico elaborato sia provvisto di tag VLAN.

Infine, il fatto che tutte le entità emulate condividano il medesimo dominio di clock (quello del sistema host) solleva una problematica fondamentale: `ptp4l` non può e non deve assolvere alla sua funzione primaria, ovvero **correggere** l'orologio locale.

Sebbene il processo possa tecnicamente applicare tali correzioni (come dimostrabile in scenari di test isolati), in una topologia complessa gli interventi effettuati da un nodo andrebbero ad alterare l'orologio letto contemporaneamente dagli altri. Questo fenomeno di interferenza (*clock fighting*) rende instabile il loop di controllo interno al protocollo, che faticherebbe a convergere e rischierebbe di rimanere bloccato in uno stato UNCALIBRATED.

Per garantire la stabilità dell'emulazione, la soluzione architetturale adottata è configurare `ptp4l` in modalità `free_running` 1: in questo modo il demone disa-

bilita le correzioni attive del clock, ma il protocollo continua a operare normalmente, scambiando regolarmente i messaggi ed eleggendo il Grandmaster tramite l'algoritmo BMCA.

3.3.4 Unidirezionalità

Come precedentemente accennato, lo scopo del prototipo è quello di garantire determinati requisiti di qualità e affidabilità in maniera strettamente unidirezionale, ovvero dal Talker verso il Listener.

In figura 3.2 è possibile osservare una coppia veth, nominata `veth-ret-repl`:`veth-ret-elim`, il cui scopo è fornire un percorso di ritorno dal Listener al Talker. Questo canale di comunicazione inverso non è infatti offerto dalle veth adibite esclusivamente alle operazioni di duplicazione ed eliminazione del traffico.

Sebbene teoricamente possibile, l'implementazione del protocollo FRER in maniera simmetrica (bidirezionale) tra i due endpoint avrebbe comportato una notevole complessità architetturale, richiedendo l'esecuzione di altre due istanze attive del programma XDP e un ulteriore incapsulamento (*wrapping*) dell'interfaccia `wlan1`.

3.4 Test e Considerazioni

3.4.1 FRER

Per validare il corretto funzionamento dell'applicazione XDP FRER sono stati condotti due test:

1. Il primo verifica che i pacchetti vengano effettivamente duplicati dal Bridge Replication ed eliminati dal Bridge Elimination.
2. Il secondo verifica che, nel caso in cui un collegamento venga meno, l'infrastruttura di comunicazione sia preservata, garantendo l'alta disponibilità del servizio.

Per il primo test, si faccia riferimento al listato 3.9. Una prima fase di inizializzazione registra il valore di partenza del contatore `xdp_xmit` sulle due interfacce `veth-repl` e `veth-repl-b` (che rappresenta il numero di pacchetti instradati verso le rispettive interfacce dal layer XDP). Successivamente, viene rilevato il valore iniziale dei contatori *Passed* e *Dropped* forniti dal programma XDP.

In seguito, viene effettuata la trasmissione di 10 pacchetti ICMP dal Talker verso il Listener tramite il comando `ping`, al termine della quale vengono nuovamente campionati i valori dei contatori al fine di poterne effettuare il confronto.

```

1 A0=$(ip netns exec bridge-repl ethtool -S veth-repl | awk
  '/xdp_xmit:/{print_$2;_exit}')
2 B0=$(ip netns exec bridge-repl ethtool -S veth-repl-b | awk
  '/xdp_xmit:/{print_$2;_exit}')
3 P0=$(grep -a 'Passed:' /tmp/frer_elim.log | tail -1 | awk
  '{print_$2}' | tr -dc 0-9) # contatore passed
4 D0=$(grep -a 'Passed:' /tmp/frer_elim.log | tail -1 | awk
  '{print_$4}' | tr -dc 0-9) # contatore dropped
5 ip netns exec talker ping -c 10 -i 0.3 -W 1 10.0.0.20 | tail -2
6 sleep 2
7 A1=$(ip netns exec bridge-repl ethtool -S veth-repl | awk
  '/xdp_xmit:/{print_$2;_exit}')
8 B1=$(ip netns exec bridge-repl ethtool -S veth-repl-b | awk
  '/xdp_xmit:/{print_$2;_exit}')
9 P1=$(grep -a 'Passed:' /tmp/frer_elim.log | tail -1 | awk
  '{print_$2}' | tr -dc 0-9) # contatore passed
10 D1=$(grep -a 'Passed:' /tmp/frer_elim.log | tail -1 | awk
  '{print_$4}' | tr -dc 0-9) # contatore dropped
11 echo di output..

```

Listing 3.9: Test di replicazione ed eliminazione XDP FRER (test/test_prototipo.sh)

È possibile osservare l'esito di questo test nel listato 3.10:

```

1 replicazione percorso A (veth-repl) : 10
2 replicazione percorso B (veth-repl-b) : 10
3 eliminazione passati al listener : 10
4 eliminazione duplicati scartati : 10

```

Listing 3.10: Output del test di replicazione ed eliminazione XDP FRER

Il test conferma che la replicazione è avvenuta con successo (su entrambi i percorsi ridondanti sono transitati 10 pacchetti) e che la fase di eliminazione ha operato correttamente, facendo sì che il Listener ricevesse esattamente i 10 pacchetti originali.

Nel listato 3.11 è presentato lo script che simula la caduta di uno dei percorsi. La logica applicata è analoga a quella del caso precedente; tuttavia, essendo i due test progettati per essere eseguiti in sequenza stretta, questo secondo script non ricalcola la baseline iniziale dei contatori, bensì sfrutta l'ultimo stato acquisito.

```

1 ip netns exec bridge-repl ip link set veth-repl down
2 sleep 1
3 A0=$A1; B0=$B1
4 ip netns exec talker ping -c 10 -i 0.3 -W 1 10.0.0.20 | tail -2
5 sleep 2
6 A1=$(ip netns exec bridge-repl ethtool -S veth-repl | awk
  '/xdp_xmit:/{print_$2;_exit}')

```

```

7  B1=$(ip netns exec bridge-repl ethtool -S veth-repl-b | awk
   '/xdp_xmit:{print_$2;_exit}')
8
9  echo di output..

```

Listing 3.11: Script per il test di rimozione di un percorso XDP FRER (test/test_prototipo.sh)

L'esito di questa seconda prova, riportato nel listato 3.12, conferma che il sistema si comporta correttamente a fronte dell'interruzione di un percorso, instradando con successo il traffico sul collegamento superstite.

```

1  percorso A (abbattuto) : 0
2  percorso B              : 10

```

Listing 3.12: Output del test in seguito alla rimozione di un percorso

3.4.2 PTP

Come discusso in precedenza, la necessità di utilizzare `ptp4l` in modalità `free_running` 1 impedisce agli slave di correggere attivamente il proprio clock e, di conseguenza, di convergere allo stato `s2` (sincronizzato). Tuttavia, è possibile validare il corretto funzionamento del protocollo a livello infrastrutturale dimostrando la corretta elezione del Grandmaster tramite l'algoritmo BMCA (*Best Master Clock Algorithm*).

A tal fine, uno dei due bridge è stato configurato intenzionalmente con una priorità maggiore (ossia un valore numerico inferiore). Nel listato 3.13 è possibile notare come il file di configurazione del Bridge Replication presenti il parametro `priority1` impostato a 100, a differenza degli altri nodi della rete che mantengono il valore di default pari a 128.

```

1  [global]
2  network_transport      L2
3  time_stamping          software
4  tx_timestamp_timeout   100
5  free_running           1
6  sanity_freq_limit      0
7  logSyncInterval        -3
8  logMinDelayReqInterval -3
9  priority1              100
10 [mgmt]
11 [ptp-repl0]

```

Listing 3.13: Configurazione `ptp4l-repl.conf` (scripts/ptp4l_conf/ptp4l-repl.conf)

Avviando il demone `ptp4l` sui quattro nodi della topologia, si osserva nel listato 3.14 che l'algoritmo BMCA elegge correttamente il Bridge Replication al ruolo di Grandmaster, rispettando la gerarchia imposta.

```

1 bridge-repl : assuming the grand master role
2 bridge-elim : selected best master clock 5e2fe8.ffff.b2a260
3 talker      : selected best master clock 5e2fe8.ffff.b2a260
4 listener    : selected best master clock 5e2fe8.ffff.b2a260
5 # MAC address del bridge-repl: 5e2fe8.ffff.b2a260

```

Listing 3.14: Output dell'elezione del Grandmaster sui 4 nodi

Infine, analizzando i log generati dal processo `ptp4l` sul Listener (listato 3.15), si ha l'ulteriore conferma che lo scambio dei messaggi previsti dal protocollo avviene regolarmente. Il demone identifica il Grandmaster e transita correttamente nello stato `s0` (UNCALIBRATED); tuttavia, operando in *free running*, non interviene sull'oscillatore di sistema. Di conseguenza, come ampiamente previsto, il loop di controllo non si chiude e parametri come il *master offset*, il *path delay* e la correzione di frequenza (*freq*) fluttuano continuamente senza mai stabilizzarsi.

```

1 ptp4l[12851.735]: port 1 (ptp-lis0): assuming the grand master
  role
2 ptp4l[12852.438]: port 1 (ptp-lis0): new foreign master
  32beac.ffff.71c254-2
3 ptp4l[12856.438]: selected best master clock 5e2fe8.ffff.b2a260
4 ptp4l[12856.439]: foreign master not using PTP timescale
5 ptp4l[12856.439]: port 1 (ptp-lis0): MASTER to UNCALIBRATED on
  RS_SLAVE
6 ptp4l[12858.443]: master offset      -33848 s0 freq  +5160 path
  delay      120762
7 ptp4l[12860.445]: master offset      -13044 s0 freq  -9157 path
  delay       81628
8 ptp4l[12862.447]: master offset      -46133 s0 freq  +8083 path
  delay      130901
9 ptp4l[12864.450]: master offset      -89257 s0 freq  -31222 path
  delay      111492
10 ptp4l[12866.452]: master offset      -93863 s0 freq   +109 path
  delay      116316
11 ptp4l[12868.454]: master offset      -76896 s0 freq  +6579 path
  delay      112521
12 ptp4l[12870.456]: master offset      -29484 s0 freq  +8003 path
  delay       81130
13 ptp4l[12872.458]: master offset     -100807 s0 freq  -15666 path
  delay      121086
14 ptp4l[12874.460]: master offset      -31053 s0 freq  +24646 path
  delay      100665
15 ptp4l[12876.463]: master offset      -43457 s0 freq  +7801 path
  delay      128689

```

```

16      ptp4l[12878.465]: master offset      -16557 s0 freq  +5777 path
      delay      113356
17      ptp4l[12880.467]: master offset      -6918 s0 freq  +16309 path
      delay      136375
18      ptp4l[12882.470]: master offset      -37909 s0 freq  -26181 path
      delay      114919
19      ptp4l[12884.472]: master offset      -26461 s0 freq  -1305 path
      delay      100858
20      ptp4l[12886.474]: master offset      -62241 s0 freq  -519 path
      delay      135599

```

Listing 3.15: Output di ptp4l sul nodo Listener

3.4.3 TAPRIO

Per validare il funzionamento di taprio sono stati sviluppati due script Python: `tsn_stream.py` e `tsn_monitor.py` (entrambi disponibili nella repository del prototipo, sotto `test/`). Il primo, riportato nel listato 3.16, viene eseguito sul nodo Talker e genera un flusso di traffico UDP basato sui parametri passati in ingresso al processo (`<ip>` `<porta>` `<priorita>` `<secondi>` `<pps>`, dove `pps` indica i pacchetti al secondo). Lo script costruisce un payload di 64 byte, inserendovi 8 byte per il numero di sequenza, 8 byte per il timestamp di invio e i restanti 48 byte come *padding*.

```

1      # uso: tsn_stream.py <ip> <porta> <priorita> <secondi> <pps>
      (pps 0 = massima velocita')
2      import socket, struct, sys, time
3
4      ip      = sys.argv[1]
5      porta   = int(sys.argv[2])
6      priorita = int(sys.argv[3])
7      secondi  = float(sys.argv[4])
8      pps      = float(sys.argv[5])
9
10     s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
11     s.setsockopt(socket.SOL_SOCKET, socket.SO_PRIORITY, priorita)
12
13     pausa = 1.0 / pps if pps > 0 else 0
14     fine  = time.time() + secondi
15     n = 0
16
17     # 64 byte di payload: 8 di sequenza, 8 di timestamp, 48 di
      riempimento
18     while time.time() < fine:
19         s.sendto(struct.pack("!QQ", n, time.time_ns()) + b"x" * 48,
      (ip, porta))
20         n += 1

```

```

21         if pausa:
22             time.sleep(pausa)
23
24     print("INVIATI_%d_pacchetti_con_SO_PRIORITY_%d_(%.0f_pkt/s)" %
          (n, priorita, n / secondi))

```

Listing 3.16: Script di generazione traffico `tsn_stream.py` (test/`tsn_stream.py`)

Il secondo script, `tsn_monitor.py` (listato 3.17), viene eseguito sul nodo Listener e ha il compito di monitorare la ricezione del flusso. Accetta in input la porta di ascolto, la durata totale del monitoraggio in secondi e la *finestra* di campionamento, ovvero l'intervallo temporale su cui calcolare e stampare periodicamente le statistiche. A livello implementativo, lo script si avvale di due strutture dati principali: un array `lat` per memorizzare le latenze dei pacchetti in ingresso e un array `seq` per tracciare i numeri di sequenza, parametro fondamentale per rilevare con esattezza l'eventuale perdita di pacchetti (*packet loss*).

```

1     # uso: tsn_monitor.py <porta> <secondi> <finestra>
2     import socket, struct, sys, time
3
4     porta      = int(sys.argv[1])
5     secondi    = float(sys.argv[2])
6     finestra   = float(sys.argv[3])
7
8     s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
9     s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
10    s.bind(("0.0.0.0", porta))
11    s.settimeout(0.2)
12
13    def stampa(t, lat, seq):
14        if not lat:
15            print("t=%5.1fs_nessun_pacchetto" % t)
16            return
17        attesi = max(seq) - min(seq) + 1
18        persi  = attesi - len(seq)
19        print("t=%5.1fs_ricevuti_%3d/%3d_persi_%3d_(%5.1f%)_min_%6.3f_avg_%6.3f_max_%6.3f_jitter_%6.3f_ms"
20              % (t, len(seq), attesi, persi, 100.0 * persi / attesi,
21                min(lat), sum(lat) / len(lat), max(lat), max(lat) -
22                min(lat)))
23
24    inizio      = time.time()
25    fine        = inizio + secondi
26    prossima    = inizio + finestra
27    lat, seq    = [], []
28    totale      = 0
29    primo       = None
30    ultimo      = None

```

```

30
31 while time.time() < fine:
32     try:
33         dati, _ = s.recvfrom(128)
34         n, ts = struct.unpack("!QQ", dati[:16])
35         lat.append((time.time_ns() - ts) / 1e6)
36         seq.append(n)
37         totale += 1
38         primo = n if primo is None else min(primo, n)
39         ultimo = n if ultimo is None else max(ultimo, n)
40     except socket.timeout:
41         pass
42     if time.time() >= prossima:
43         stampa(prossima - inizio, lat, seq)
44         lat, seq = [], []
45         prossima += finestra
46
47 if primo is not None:
48     attesi = ultimo - primo + 1
49     persi = attesi - totale
50     print("TOTALE_____ricevuti_{}_{}_persi_{}_({}.1f%%)"
51           % (totale, attesi, persi, 100.0 * persi / attesi))

```

Listing 3.17: Script di monitoraggio tsn_monitor.py (test/tsn_monitor.py)

Lo scenario di test, il cui script di automazione è riportato nel listato 3.18, prevede l'avvio iniziale di un flusso critico ad alta priorità. Successivamente, per simulare una condizione di forte stress di rete, viene iniettato un secondo flusso *best-effort* (a bassa priorità ma ad altissima frequenza di trasmissione), avente lo scopo di saturare il mezzo trasmissivo. Al fine di ottenere un termine di paragone oggettivo, l'esperimento è stato eseguito in due configurazioni distinte: la prima utilizzando l'algoritmo di schedulazione standard *pfi fo_fast* (la *queuing discipline* di default applicata dal kernel alle interfacce), e la seconda applicando *taprio*, precedentemente configurato come mostrato nel listato 3.6.

È fondamentale precisare che, per rendere i risultati dell'emulazione metricamente significativi, è stato introdotto artificialmente un collo di bottiglia sulla banda del link (tramite l'impiego del *qdisc netem rate 10mbit*). In assenza di tale limitazione, operando all'interno di un ambiente emulato basato su namespace e interfacce virtuali, il throughput complessivo sarebbe stato vincolato unicamente dalla larghezza di banda della memoria RAM del sistema host; di conseguenza, il traffico interferente non sarebbe mai riuscito a generare una reale congestione delle code di trasmissione.

```

1 ip netns exec bridge-elim tc qdisc replace dev wlan2 root netem
  rate 10mbit limit 100
2

```

```

3  echo "_---_con_pfifo_fast_---"
4  ip netns exec bridge-repl tc qdisc replace dev vproxy-repl-in
   root pfifo_fast
5  sleep 1
6  ip netns exec listener stdbuf -oL python3 tsn_monitor.py 5000 24
   4 &
7  MON=$!
8  sleep 1
9  ip netns exec talker python3 tsn_stream.py 10.0.0.20 5000 3 24
   20 >/dev/null 2>&1 &
10 CRIT=$!
11 sleep 11
12 echo ">>>_parte_1'interferenza_best_effort,_a_massima_velocita'"
13 ip netns exec talker python3 tsn_stream.py 10.0.0.20 5001 0 8 0
   >/dev/null 2>&1
14 echo ">>>_interferenza_terminata"
15 ..
16 ..
17 ..
18 echo "atteso:_80_pacchetti_critici_per_finestra_da_4_s"
19 echo "---_con_taprio_---"
20 ip netns exec listener stdbuf -oL python3 tsn_monitor.py 5000 24
   4 &
21 MON=$!
22 sleep 1
23 ip netns exec talker python3 tsn_stream.py 10.0.0.20 5000 3 24
   20 > /tmp/tsn_test/critico.txt 2>&1 &
24 CRIT=$!
25 sleep 11
26 echo ">>>_parte_1'interferenza_best_effort,_a_massima_velocita'"
27 ip netns exec talker python3 tsn_stream.py 10.0.0.20 5001 0 8 0
   > /tmp/tsn_test/flood.txt 2>&1
28 echo ">>>_interferenza_terminata"

```

Listing 3.18: Avvio dei test con e senza taprio (test/test_prototipo.sh)

Analizzando l'output riportato nel listato 3.19, è possibile valutare il comportamento del sistema durante gli 8 secondi in cui viene iniettato il traffico interferente (*background traffic*) a bassa priorità. In entrambi gli scenari si registra fisiologicamente un incremento della latenza e del jitter, ma il confronto diretto evidenzia un paradigma essenziale delle reti TSN: l'importanza cruciale della schedulazione deterministica ai fini dell'affidabilità della comunicazione.

Osservando i grafici riportati in Figura 3.5 e 3.6, l'impatto del protocollo taprio risulta evidente e drastico. Con la coda standard `pfifo_fast`, il flusso critico subisce immediatamente gli effetti di *starvation* causati dall'enorme mole di pacchetti *best-effort*, arrivando a registrare picchi di *packet loss* superiori al 46% durante la finestra di congestione, e attestandosi su una perdita complessiva del 15.9% sul-

l'intero esperimento.

Al contrario, l'impiego di *taprio* isola temporalmente il flusso critico, assegnandogli finestre di trasmissione (*time slots*) dedicate ed esclusive. Questo rigoroso meccanismo di *gating* protegge i pacchetti ad alta priorità dalla congestione sottostante, abbattendo il picco di perdita all'1.3% nel momento di massimo stress e riducendo il *packet loss* totale a un trascurabile 0.2%.

```
1  --- con pfifo_fast ---
2  >>> parte l'interferenza_best_effort, _a_massima_velocita'
3  >>> interferenza terminata
4  t= 4.0s   ricevuti 61/ 61   persi    0 ( 0.0%)   min 0.292
avg 0.574   max 0.793   jitter 0.501 ms
5  t= 8.0s   ricevuti 79/ 79   persi    0 ( 0.0%)   min 0.270
avg 0.623   max 1.006   jitter 0.736 ms
6  t= 12.0s  ricevuti 79/ 79   persi    0 ( 0.0%)   min 0.278
avg 0.693   max 1.065   jitter 0.787 ms
7  t= 16.0s  ricevuti 43/ 80   persi   37 (46.2%)   min 8.500
avg 8.582   max 8.657   jitter 0.157 ms
8  t= 20.0s  ricevuti 44/ 80   persi   36 (45.0%)   min 4.562
avg 8.497   max 8.646   jitter 4.084 ms
9  t= 24.0s  ricevuti 79/ 79   persi    0 ( 0.0%)   min 0.333
avg 0.679   max 0.846   jitter 0.513 ms
10 TOTALE      ricevuti 385/458   persi 73 (15.9%)
11
12 --- con taprio ---
13 >>> parte l'interferenza_best_effort, _a_massima_velocita'
14 >>> interferenza terminata
15 t= 4.0s   ricevuti 60/ 60   persi    0 ( 0.0%)   min 0.223
avg 0.667   max 0.850   jitter 0.627 ms
16 t= 8.0s   ricevuti 80/ 80   persi    0 ( 0.0%)   min 0.267
avg 0.676   max 0.992   jitter 0.725 ms
17 t= 12.0s  ricevuti 79/ 79   persi    0 ( 0.0%)   min 0.240
avg 0.770   max 6.476   jitter 6.236 ms
18 t= 16.0s  ricevuti 78/ 79   persi    1 ( 1.3%)   min 7.739
avg 8.263   max 9.176   jitter 1.437 ms
19 t= 20.0s  ricevuti 80/ 80   persi    0 ( 0.0%)   min 7.732
avg 8.255   max 9.193   jitter 1.461 ms
20 t= 24.0s  ricevuti 79/ 79   persi    0 ( 0.0%)   min 0.276
avg 0.676   max 1.043   jitter 0.767 ms
21 TOTALE      ricevuti 456/457   persi 1 (0.2%)
22 INVIATI 477 pacchetti con SO_PRIORITY 3 (20 pkt/s)
23 INVIATI 309660 pacchetti con SO_PRIORITY 0 (38708 pkt/s)
```

Listing 3.19: Output dei test con e senza *taprio*

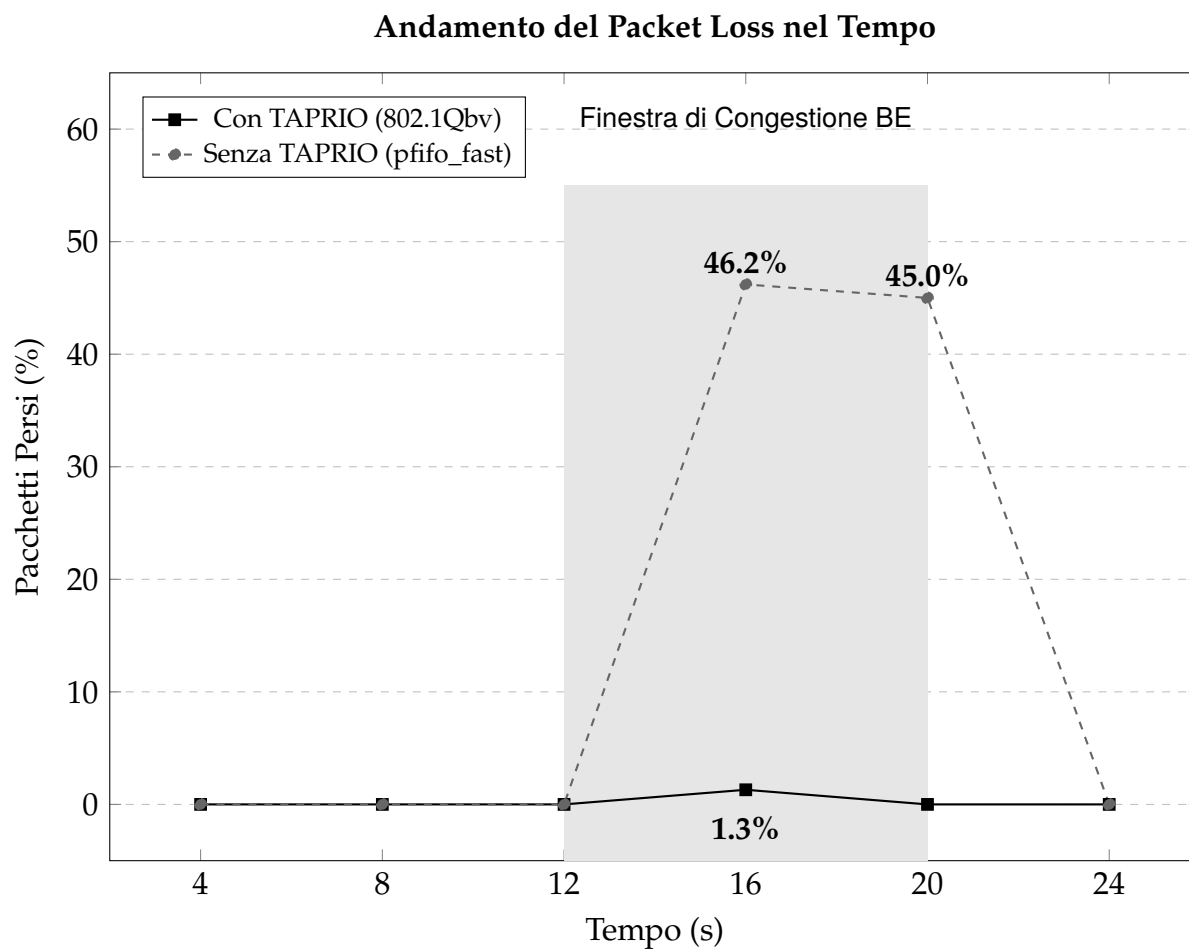


Figura 3.5: Confronto del packet loss nel tempo tra TAPRIO e coda standard (pfifo_fast) durante un'iniezione di traffico *best-effort*.

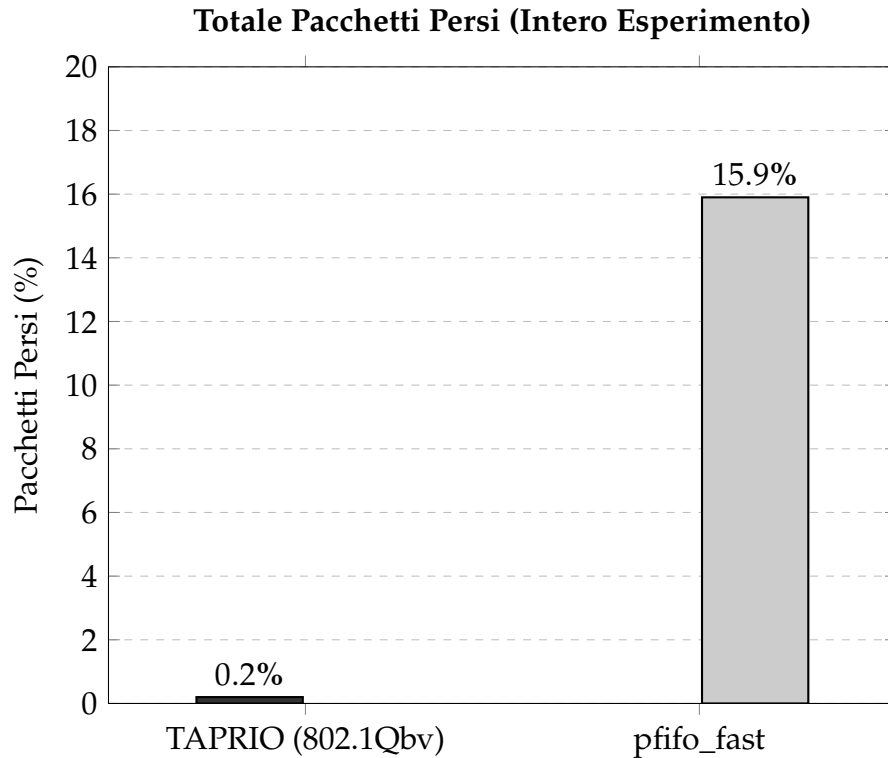


Figura 3.6: Percentuale complessiva di pacchetti critici persi sull'intero periodo di test.

3.4.4 Conclusioni

In conclusione, l'esito positivo dei test condotti permette di validare l'architettura del prototipo nella sua interezza. Nonostante le molteplici sfide implementative derivanti dai vincoli dell'ambiente virtualizzato — quali l'assenza di code di trasmissione multiple nell'emulatore `mac80211_hwsim`, l'impossibilità di sfruttare l'*hardware timestamping* per la convergenza del protocollo PTP e la necessità di ricorrere a complessi incapsulamenti logici per far coesistere l'elaborazione eBPF/XDP con il *Traffic Control* — i risultati dimostrano la concreta fattibilità dell'estensione dei meccanismi TSN su collegamenti Wi-Fi 7.

È ragionevole affermare che la trasposizione di questo *proof of concept* su dispositivi di rete fisici, dotati di supporto nativo alle specifiche Wi-Fi 7 e TSN, farebbe decadere la quasi totalità dei *workaround* architetturali resisi necessari in questa sede. L'impiego di NIC (*Network Interface Card*) reali, provviste di *Physical Hardware Clock* indipendenti, code multiple fisiche e implementazioni hardware del sottolivello MAC per la *frame preemption*, non solo snellirebbe drasticamente la configurazione dello stack di rete, ma permetterebbe al sistema di esprimere appieno il proprio

potenziale deterministico, confermando la solidità della convergenza IT/OT per gli scenari industriali di nuova generazione.

Riferimenti bibliografici

- [1] Toni Adame, Marc Carrascosa e Boris Bellalta. *Time-Sensitive Networking in IEEE 802.11be: On the Way to Low-latency WiFi 7*. 2020. arXiv: 1912.06086 [cs.NI]. URL: <https://arxiv.org/abs/1912.06086>.
- [2] Aishwarya Choorakuzhiyil, Kevin Ho e Sara Reyes. *Comparison of STR and EMLSR Performance in Wi-Fi 7 MLO*. 2025. arXiv: 2501.04149 [cs.NI].
- [3] Cailian Deng, Xuming Fang, Xiao Han, Xianbin Wang, Li Yan, Rong He, Yan Long e Yuchen Guo. "IEEE 802.11be Wi-Fi 7: New Challenges and Opportunities". In: *IEEE Communications Surveys Tutorials* 22.4 (2020), pp. 2136–2166.
- [4] Jorge Luis Diaz-Martinez, Emiro De-La-Hoz-Franco, Johan David Mardini Bovea, Shariq Aziz Butt e Tauseef Jamal. "Software Defined Networking for Organizations Network Automation". In: ().
- [5] Ferenc Fejes, Ferenc Orosi, Balázs Varga e János Farkas. *Lightweight Implementation of Per-packet Service Protection in eBPF/XDP*. 2023. arXiv: 2312.07152 [cs.NI].
- [6] Norman Finn. "Introduction to Time-Sensitive Networking". In: *IEEE Communications Standards Magazine* 6.4 (2022), pp. 8–13. DOI: 10.1109/MCOMSTD.0004.2200046.
- [7] Yoshifumi Hotta, Atsushi Inoue, Hisakazu Bessho, Christophe Mangin e Ryusuke Kawate. "Experimental Study of a Low-Delay Ethernet Switch for Real Time Networks". In: 2015. URL: <https://api.semanticscholar.org/CorpusID:204791926>.
- [8] "IEEE Standard for Local and Metropolitan Area Networks–Bridges and Bridged Networks". In: *IEEE Std 802.1Q-2022 (Revision of IEEE Std 802.1Q-2018)* (2022), pp. 1–2163. DOI: 10.1109/IEEESTD.2022.10004498.
- [9] *IEEE Std 802.3br-2016: IEEE Standard for Ethernet – Amendment 5: Specification and Management Parameters for Interspersing Express Traffic*. New York, NY, USA: IEEE Computer Society, 2016.
- [10] Intel. *Configuring TSN Qdiscs*. URL: <https://tsn.readthedocs.io/qdiscs.html>.

- [11] Intel. *Configuring VLAN Interfaces*. URL: <https://tsn.readthedocs.io/vlan.html>.
- [12] V. Jacobson. "Congestion avoidance and control". In: *Symposium Proceedings on Communications Architectures and Protocols*. SIGCOMM '88. Stanford, California, USA: Association for Computing Machinery, 1988, pp. 314–329. ISBN: 0897912799.
- [13] Abdalla Mahamid, Filip Rezabek e Kilian Holzinger. "Time Sensitive Networking - 802.1Qci". In: ().
- [14] Lisa Maile, Dominik Voitlein, Kai-Steffen Hielscher e Reinhard German. "Ensuring Reliable and Predictable Behavior of IEEE 802.1CB Frame Replication and Elimination". In: *ICC 2022 - IEEE International Conference on Communications*. 2022, pp. 2706–2712.
- [15] Sugandh Huthanahally Mohan e Rute C. Sofia. "Fine Time Measurement Based Synchronization for Industrial Wireless/Wired Networks". In: *2022 25th International Symposium on Wireless Personal Multimedia Communications (WPMC)*. 2022, pp. 357–362. DOI: 10.1109/WPMC55625.2022.10014703.
- [16] Erfan Mozaffariahrar, Francesc Wilhelmi, Lorenzo Galati-Giordano, Pasquale Imputato, Michael Menth e Stefano Avallone. "R-TWT in Wi-Fi 7 and Beyond: Enabling Bounded Latency, Energy Efficiency, and Reliability". In: *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*. 2025, pp. 1–8. DOI: 10.1109/ETFA65518.2025.11205686.
- [17] mwarning. *Why and How to Use Netlink Socket*. URL: https://github.com/mwarning/netlink-examples/blob/master/articles/Why_and_How_to_Use_Netlink_Socket.md.
- [18] Ahmed Nasrallah, Akhilesh Thyagaturu, Ziyad Alharbi, Cuixiang Wang, Xing Shao, Martin Reisslein e Hesham Elbakoury. "Performance comparison of IEEE 802.1 TSN time aware shaper (TAS) and asynchronous traffic shaper (ATS)". In: *IEEE Access* PP (apr. 2019), pp. 1–1.
- [19] Computer Networking Notes. *VLAN Tagging*. URL: <https://www.computernetworkingnotes.com>.
- [20] Cheng Pei-Chi. *Time Synchronization with Linux Precision Time Protocol*. URL: <https://free5gc.org/blog/20250709/20250709/>.
- [21] Christopher Pfefferle, Florian Wiedner e Christoph Schwarzenberg. "IEEE 802.1Qcr Asynchronous Traffic Shaping with Linux Traffic Control". In: .
- [22] Boxin Shi, Xiaodong Tu, Bin Wu e Yifei Peng. "Recent Advances in Time-Sensitive Network Configuration Management: A Literature Review". In: *Journal of Sensor and Actuator Networks* 12.4 (2023). ISSN: 2224-2708.
- [23] Luis Silva, Paulo Pedreiras, Pedro Fonseca e Luis Almeida. "On the adequacy of SDN and TSN for Industry 4.0". In: *2019 IEEE 22nd International Symposium on Real-Time Distributed Computing (ISORC)*.

- [24] Philip Smith e Barry Greene. *Introduction to IS-IS*. Workshop Presentation. Licensed under CC BY-NC 4.0. Available at <http://www.bgp4all.com>. E-mail: workshop@bgp4all.com.
- [25] WafaiCloud Team. *Setting Up a VLAN on Linux Using iproute2*. URL: <https://wafaicloud.com/blog/setting-up-a-vlan-on-linux-using-iproute2/>.
- [26] Tektronix. *Wi-Fi: Overview of the 802.11 Physical Layer*. URL: https://download.tek.com/document/37W-29447-2_LR.pdf.
- [27] TSN. *Time-Sensitive Networking*. URL: <https://1.ieee802.org/tsn/>.
- [28] Tianyu Zhang, Gang Wang, Chuanyu Xue, Jiachen Wang, Mark Nixon e Song Han. *Time-Sensitive Networking (TSN) for Industrial Automation: Current Advances and Future Directions*. 2024. arXiv: 2306.03691 [cs.NI].

Appendici

Appendice A

Ambiente docker per test Linux PTP

```
#!/usr/bin/env bash
# =====
# docker_network_setup.sh
# =====

set -euo pipefail

# ----- Configurazione -----
NETWORK_NAME="mynet"
CONTAINER_1="node1"
CONTAINER_2="node2"
CONTAINER_3="node3"
IMAGE="alpine:latest"

# Colori per output
GREEN='\033[0;32m'
CYAN='\033[0;36m'
RED='\033[0;31m'
NC='\033[0m' # No Color

log() { echo -e "${CYAN}[INFO]${NC}__${*}"; }
ok() { echo -e "${GREEN}[OK]${NC}____${*}"; }
err() { echo -e "${RED}[ERROR]${NC}__${*}" >&2; exit 1; }

# ----- Pulizia preventiva -----
cleanup() {
    log "Pulizia_risorse_esistenti_(se_presenti)..."
    docker rm -f "$CONTAINER_1" "$CONTAINER_2" "$CONTAINER_3" 2>/dev/null && ok
    "Container_rimossi" || true
    docker network rm "$NETWORK_NAME" 2>/dev/null && ok "Rete_rimossa"
    || true
}

cleanup

# ----- 1. Crea la rete bridge -----
log "Creazione_rete_Docker:__$NETWORK_NAME"
docker network create \
    --driver bridge \
    --subnet 172.20.0.0/24 \
```

```

--gateway 172.20.0.1 \
"$NETWORK_NAME"
ok "Rete_'$NETWORK_NAME'_creata_(172.20.0.0/24)"

# ----- 2. Avvia i container -----
log "Avvio_container:_$CONTAINER_1"
docker run -d \
--cap-add=NET_ADMIN \
--cap-add=SYS_TIME \
--name "$CONTAINER_1" \
--network "$NETWORK_NAME" \
--ip 172.20.0.10 \
"$IMAGE" \
sh -c "while true;do sleep 3600;done"
ok "$CONTAINER_1_avviato_(IP:_172.20.0.10)"

log "Avvio_container:_$CONTAINER_2"
docker run -d \
--cap-add=NET_ADMIN \
--cap-add=SYS_TIME \
--name "$CONTAINER_2" \
--network "$NETWORK_NAME" \
--ip 172.20.0.11 \
"$IMAGE" \
sh -c "while true;do sleep 3600;done"
ok "$CONTAINER_2_avviato_(IP:_172.20.0.11)"

log "Avvio_container:_$CONTAINER_3"
docker run -d \
--cap-add=NET_ADMIN \
--cap-add=SYS_TIME \
--name "$CONTAINER_3" \
--network "$NETWORK_NAME" \
--ip 172.20.0.12 \
"$IMAGE" \
sh -c "while true;do sleep 3600;done"
ok "$CONTAINER_3_avviato_(IP:_172.20.0.12)"

# ----- 3. Verifica connettività -----
log "Test_ping:_$CONTAINER_1_-$CONTAINER_2(per_nome)"
docker exec "$CONTAINER_1" ping -c 3 "$CONTAINER_2" \
&& ok "Ping_riuscito:_$CONTAINER_1_-$CONTAINER_2" \
|| err "Ping_fallito!"

log "Test_ping:_$CONTAINER_2_-$CONTAINER_1(per_IP)"
docker exec "$CONTAINER_2" ping -c 3 172.20.0.10 \
&& ok "Ping_riuscito:_$CONTAINER_2_-$CONTAINER_1" \
|| err "Ping_fallito!"

log "Test_ping:_$CONTAINER_3_-$CONTAINER_1(per_IP)"
docker exec "$CONTAINER_3" ping -c 3 172.20.0.10 \
&& ok "Ping_riuscito:_$CONTAINER_3_-$CONTAINER_1" \
|| err "Ping_fallito!"

log "Test_ping:_$CONTAINER_3_-$CONTAINER_2(per_IP)"
docker exec "$CONTAINER_3" ping -c 3 172.20.0.11 \
&& ok "Ping_riuscito:_$CONTAINER_3_-$CONTAINER_2" \
|| err "Ping_fallito!"

# ----- 4. Installo tool necessari -----
log "Install_bash_su_$CONTAINER_1_e_$CONTAINER_2_e_$CONTAINER_3"

```

```

docker exec "$CONTAINER_1" apk add --no-cache bash \
    && ok "Bash_installato_su_$CONTAINER_1" \
    || err "Installazione_di_bash_fallita_su_$CONTAINER_1"

docker exec "$CONTAINER_2" apk add --no-cache bash \
    && ok "Bash_installato_su_$CONTAINER_2" \
    || err "Installazione_di_bash_fallita_su_$CONTAINER_2"

docker exec "$CONTAINER_3" apk add --no-cache bash \
    && ok "Bash_installato_su_$CONTAINER_3" \
    || err "Installazione_di_bash_fallita_su_$CONTAINER_3"

log "Abilito_repository_di_testing_su_$CONTAINER_1_e_$CONTAINER_2_e_$CONTAINER_3"

docker exec "$CONTAINER_1" sh -c \
    'echo "@testing https://dl-cdn.alpinelinux.org/alpine/edge/testing" >> \
    /etc/apk/repositories' \
    && ok "Repository_di_testing_abilitato_su_$CONTAINER_1" \
    || err "Abilitazione_repository_di_testing_fallita_su_$CONTAINER_1"

docker exec "$CONTAINER_2" sh -c \
    'echo "@testing https://dl-cdn.alpinelinux.org/alpine/edge/testing" >> \
    /etc/apk/repositories' \
    && ok "Repository_di_testing_abilitato_su_$CONTAINER_2" \
    || err "Abilitazione_repository_di_testing_fallita_su_$CONTAINER_2"

docker exec "$CONTAINER_3" sh -c \
    'echo "@testing https://dl-cdn.alpinelinux.org/alpine/edge/testing" >> \
    /etc/apk/repositories' \
    && ok "Repository_di_testing_abilitato_su_$CONTAINER_3" \
    || err "Abilitazione_repository_di_testing_fallita_su_$CONTAINER_3"

log "Installo_linuxptp_su_$CONTAINER_1_e_$CONTAINER_2_e_$CONTAINER_3"

docker exec "$CONTAINER_1" sh -c \
    'apk add --no-cache linuxptp \
    --repository=https://dl-cdn.alpinelinux.org/alpine/edge/testing \
    --allow-untrusted' \
    && ok "linuxptp_installato_su_$CONTAINER_1" \
    || err "Installazione_di_linuxptp_fallita_su_$CONTAINER_1"

docker exec "$CONTAINER_2" sh -c \
    'apk add --no-cache linuxptp \
    --repository=https://dl-cdn.alpinelinux.org/alpine/edge/testing \
    --allow-untrusted' \
    && ok "linuxptp_installato_su_$CONTAINER_2" \
    || err "Installazione_di_linuxptp_fallita_su_$CONTAINER_2"

docker exec "$CONTAINER_3" sh -c \
    'apk add --no-cache linuxptp \
    --repository=https://dl-cdn.alpinelinux.org/alpine/edge/testing \
    --allow-untrusted' \
    && ok "linuxptp_installato_su_$CONTAINER_3" \
    || err "Installazione_di_linuxptp_fallita_su_$CONTAINER_3"

# ----- 5. Creo alias shell -----
cat > /tmp/docker_aliases.sh <<EOF
alias node1="docker exec -it $CONTAINER_1 bash"
alias node2="docker exec -it $CONTAINER_2 bash"
alias node3="docker exec -it $CONTAINER_3 bash"
EOF

```

```

echo ""
ok "Esegui: source /tmp/docker_aliases.sh"
# ----- 6. Copio file di configurazione linuxptp -----
log "Copio file di configurazione linuxptp per $CONTAINER_1, $CONTAINER_2 e $CONTAINER_3"

docker cp oc.conf $CONTAINER_1:/etc/ptp4l.conf
docker cp oc.conf $CONTAINER_2:/etc/ptp4l.conf
docker cp gm.conf $CONTAINER_3:/etc/ptp4l.conf

# ----- 7. Riepilogo -----
echo ""
echo "=====
echo "Riepilogo infrastruttura"
echo "=====
docker network inspect "$NETWORK_NAME" \
  --format '{{.Name}}({{.Driver}}) Subnet: {{range .IPAM.Config}}{{.Subnet}}{{end}}'
docker inspect "$CONTAINER_1" \
  --format '{{.NetworkSettings.Networks.$NETWORK_NAME}.IPAddress}}'
docker inspect "$CONTAINER_2" \
  --format '{{.NetworkSettings.Networks.$NETWORK_NAME}.IPAddress}}'
docker inspect "$CONTAINER_3" \
  --format '{{.NetworkSettings.Networks.$NETWORK_NAME}.IPAddress}}'
echo "=====
echo ""
ok "Setup completato! I container sono attivi e comunicano."
echo ""
echo "Connetti al node1: docker exec -it $CONTAINER_1 bash"
echo "Connetti al node2: docker exec -it $CONTAINER_2 bash"
echo "Connetti al node3: docker exec -it $CONTAINER_3 bash"
echo "Pulisci tutto: docker rm -f $CONTAINER_1 $CONTAINER_2 $CONTAINER_3 && docker network rm $NETWORK_NAME"

```

Appendice B

Macchina Host e Ambiente

Per favorire la replicabilità e la portabilità del prototipo, si è scelto di implementare l'intero sistema all'interno di una macchina virtuale gestita tramite Vagrant, configurata attraverso il file `Vagrantfile` e inizializzata mediante lo script `provision.sh`.

La natura innovativa dei protocolli utilizzati nel prototipo ha richiesto l'impiego di un ambiente di sviluppo di ultima generazione e, nello specifico, di un kernel Linux *bleeding edge* nella versione 7.1.4, rilasciata il 18 luglio 2026, in grado di supportare lo standard Wi-Fi 7. Nello script sono presenti le istruzioni per l'installazione di diversi tool, tra cui la *toolchain* eBPF/XDP.

Appendice C

Configurazione strumenti wireless

La versione di `hostapd` e `wpa_supplicant` che è possibile ottenere tramite *package manager* come APT si ferma alla 2.10; pertanto, per avere un supporto preliminare alle principali *features* del protocollo 802.11be, è stato necessario scaricare i sorgenti ufficiali, configurandoli e compilandoli manualmente, come mostrato nel Listato C.1.

```
ls /usr/local/src/hostap/.git >/dev/null 2>&1 || git clone --depth 1 --branch
    hostap_2_11 https://w1.fi/hostap.git /usr/local/src/hostap

cat > /usr/local/src/hostap/hostapd/.config <<'EOF'
CONFIG_DRIVER_NL80211=y
CONFIG_LIBNL32=y
CONFIG_CTRL_IFACE=y
CONFIG_CTRL_IFACE_UNIX=y
CONFIG_IEEE80211N=y
CONFIG_IEEE80211AC=y
CONFIG_IEEE80211AX=y
CONFIG_IEEE80211BE=y
CONFIG_SAE=y
CONFIG_IEEE80211W=y
CONFIG_ACS=y
EOF

make -C /usr/local/src/hostap/hostapd -j"$(nproc)"
install -m 0755 /usr/local/src/hostap/hostapd/hostapd /usr/local/sbin/hostapd
install -m 0755 /usr/local/src/hostap/hostapd/hostapd_cli /usr/local/sbin/hostapd_cli

cat > /usr/local/src/hostap/wpa_supplicant/.config <<'EOF'
CONFIG_DRIVER_NL80211=y
CONFIG_LIBNL32=y
CONFIG_CTRL_IFACE=y
CONFIG_CTRL_IFACE_UNIX=y
CONFIG_SAE=y
CONFIG_IEEE80211W=y
CONFIG_BACKEND=file
EOF

make -C /usr/local/src/hostap/wpa_supplicant -j"$(nproc)"
```

```
install -m 0755 /usr/local/src/hostap/wpa_supplicant/wpa_supplicant  
        /usr/local/sbin/wpa_supplicant  
install -m 0755 /usr/local/src/hostap/wpa_supplicant/wpa_cli /usr/local/sbin/wpa_cli
```

Listing C.1: ./scripts/hostap.sh

Appendice D

Tool Linux per il networking

D.1 veth

Le *virtual Ethernet* (veth) sono interfacce di rete virtuali fornite dal kernel Linux, create sempre in coppia e collegate tra loro come un cavo di rete punto-punto: ogni pacchetto trasmesso su uno dei due endpoint viene ricevuto dall'altro. Sono comunemente utilizzate per collegare tra loro due network namespace distinti, oppure per collegare un namespace allo stack di rete predefinito (*root namespace*). Una coppia veth viene creata tramite il comando:

```
ip link add <veth-a> type veth peer name <veth-b>
```

È inoltre possibile specificare, in fase di creazione, il numero di code di trasmissione e ricezione (*numtxqueues/numrxqueues*), caratteristica sfruttata nel prototipo per emulare il supporto a code multiple non disponibile sulle interfacce radio emulate. Le interfacce veth supportano inoltre l'offloading di programmi eBPF/XDP e possono essere utilizzate in combinazione con il sottosistema tc per redirigere e manipolare il traffico tra namespace differenti.

D.2 network namespace

I *network namespace* sono un meccanismo di isolamento offerto dal kernel Linux che permette di replicare uno stack di rete indipendente all'interno dello stesso sistema operativo: ogni namespace dispone di un proprio set di interfacce di rete, tabelle di routing, regole di firewall e socket, isolati da quelli degli altri namespace. Un nuovo namespace può essere creato ed elencato tramite i comandi:

```
ip netns add <nome>  
ip netns list
```

Per eseguire un comando all'interno di un namespace specifico si utilizza:

```
ip netns exec <nome> <comando>
```

Un'interfaccia di rete può essere spostata da un namespace all'altro tramite il comando `ip link set <interfaccia> netns <nome>`, operazione utilizzata ad esempio per assegnare uno dei due endpoint di una coppia veth (si veda D.1) al namespace di destinazione. Nel prototipo sviluppato, i network namespace sono stati impiegati per emulare in un'unica macchina host i diversi nodi della topologia di rete (talker, listener e i due bridge), condividendo però lo stesso kernel e lo stesso dominio di clock, con le conseguenze discusse in 3.3.

Ringraziamenti

Grazie a tutti